第 6 章 软件安全

本章学习要点：

- 了解软件系统所面临的安全威胁；
- 掌握软件安全开发原则和安全编码策略；
- 了解恶意代码的分类、原理，理解恶意代码分析技术；
- 了解软件测试方法分类，掌握软件安全测试基本工作原理；
- 了解软件知识产权保护法律和软件版权保护技术。

软件是为了解决生产生活实际问题而开发的完成特定功能的计算机程序，在信息化时代的社会中软件应用范围日益广泛，软件发挥的作用越来越大，担负的责任也越来越重要。软件一旦出现设计上的错误、缺陷或是漏洞，将严重威胁信息系统安全，甚至带来灾难性的影响。软件安全既是软件开发中不可或缺的部分，同时也是保障软件系统安全的关键技术和手段。通常意义上，软件安全指在保证软件可用性、可靠性、可维护性的基础上，能够保护软件本身和软件运行环境不受恶意攻击、不被非法使用及不经未授权访问。

6.1 软件安全概述

信息化是当今世界发展的大趋势，是推动经济社会变革的重要力量，软件则是信息化建设的重要组成部分。全球经济的每一部分，包括能源、运输、财政、邮政、银行、电信、公共健康、应急服务、水利、电力、工业、农业等，都依赖于计算机软件。

随着新一代信息技术加速渗透到经济和社会生活的各个领域，软件产业呈现出网络化、服务化、平台化、融合化新趋势。目前，新一代信息技术正在转向软件主导，软件在信息产业中的贡献不断增加。《中国制造 2025》《积极推进“互联网＋”行动的指导意见》和《加快推进网络信息技术自主创新》等国策的深入推进和落实，对产业变革产生深远影响，国民经济各个领域对软件产业的需求将更加强劲，尤其是对操作系统、数据库等基础软件、行业应用软件、大数据软件将产生更高、更广泛的需求。

在信息化建设过程中，软件系统的安全体系是一个复杂的体系，系统软件、应用软件和第三方软件的安全问题，以及开发、部署过程中的安全问题，如果处理不当或不加防范，都可能给整个系统带来巨大的灾难。

6.1.1 软件安全的概念

软件安全(software security)指软件在受到恶意攻击的情形下依然能够继续正确运行，并确保软件在授权范围内被合法使用。

任何软件，不论它看起来多么安全，其中都隐藏有漏洞。软件安全的目的是尽可能消除软

件漏洞，确保软件在被恶意攻击下仍然正常运行。

软件面临的主要安全威胁如图 6.1 所示。其中，安全漏洞是软件产生安全问题的根源。一方面，软件漏洞可能会造成软件在运行过程中出现错误结果或运行不稳定、崩溃等现象，甚至引起计算机死机等情况。另一方面，软件漏洞会被黑客发现、利用，进而实施窃取隐私信息，甚至破坏系统等攻击行为。

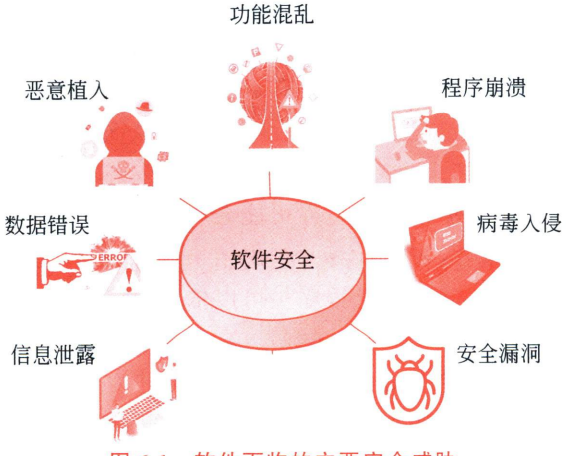


图 6.1 软件面临的主要安全威胁

软件安全即是要保护软件的完整性、可用性、保密性及运行安全性，具体如下。

(1) 软件自身安全：防止软件丢失、被破坏、被篡改、被伪造。

(2) 软件存储安全：可靠存储、保密存储、压缩存储、备份存储。

(3) 软件通信安全：安全传输、加密传输、网络安全下载、完整下载。

(4) 软件使用安全：区分合法用户与非法用户，授权访问，防止软件滥用、防止软件窃取，防止软件的非法复制。

(5) 软件运行安全：确保软件正常运行，功能正常。

为此，软件安全主要研究如何设计、构造、验证和维护软件，以保证软件是安全的，包括改进和实现软件安全的架构或结构，改进和实现软件安全的工具，以及改进或实现软件安全的方法。

6.1.2 软件缺陷和漏洞

软件安全是计算机安全中的一个关键问题。软件的缺陷包括实现中的错误(如缓冲区溢出)，以及设计中的错误(如不周全的错误处理)，且已经存在很多年了。同时，黑客常常通过利用软件漏洞入侵到系统中。因此，近年来基于互联网的应用软件往往成为风险最高的软件。同时，随着软件系统数量的不断增加和功能的复杂化，其潜在的安全隐患也不断增多。据统计，软件中的安全漏洞正逐年增长。

1. 安全漏洞的概念

安全漏洞(security hole，简称漏洞)又叫脆弱性(vulnerability)，是计算机系统具有的某种可能被入侵者恶意利用的属性。1947 年，冯·诺依曼建立计算机系统结构理论时就曾提及："计算机的发展和自然生命有相似性，一个计算机系统也有天生的类似基因的缺陷，在使用和发展过程中可能产生意想不到的问题"。20 世纪 80 年代早期黑客的出现和第一个计算机病毒的产生，使得软件漏洞逐渐引起人们的关注。在历经 40 多年的研究过程中，学术界及产业

界对漏洞给出了很多定义，漏洞定义本身也随着信息技术的发展而具有不同的含义与范畴。软件漏洞通常被认为是软件生命周期中与安全相关的设计错误、编码缺陷及运行故障等。

2. 漏洞产生的原因

漏洞的本质在于漏洞是系统的一组特征，恶意的主体（攻击者或攻击程序）能够利用这组特性，通过已授权的手段和方式获取对资源的未经授权访问，或者对系统造成损害。漏洞产生的原因如下。

（1）计算机系统结构决定了漏洞的必然性。冯·诺依曼体系结构奠定了现代计算机的基本结构，但是这一结构也导致了漏洞的产生：第一，计算机中执行的指令和处理的数据都采用二进制表示，存在指令是数据、数据也可以是指令的问题，从而导致指令可被数据篡改（如病毒感染），外部数据可被当作指令植入（如SQL注入、木马植入）；第二，由于指令和数据组成程序存储到计算机内存自动执行，数据区域的越界可能会影响指令的执行和控制（如缓冲区溢出攻击）；第三，由于程序接收外部输入进行计算并输出结果，程序的行为取决于程序员编码逻辑与外部输入数据驱动的分支路径选择，因此程序员可以依据自己的意志实现特殊功能，然后通过输入数据触发特定分支（如后门、业务逻辑漏洞）。

（2）软件趋向大型化，第三方拓展增多。常用大型软件为了充分使软件功能得到扩充，通常会有第三方拓展，这些拓展插件的存在，增加系统功能的同时也导致了安全隐患的存在，研究表明，代码行数越多，漏洞也就越多。

（3）软件新技术、新应用产生之初缺乏安全意识。比如，大多数网络协议，在设计之初就没有考虑过其安全性。当今互联网技术蓬勃发展，新技术的不断出现，也带来了大量新的安全挑战。

（4）软件使用场景更具威胁。现今软件被用于各行各业，遍及各个社会层次。软件开发者需要考虑的问题更多，并且黑客与恶意攻击者比以往有更多的机会和时间来访问软件系统，并尝试寻找、利用软件漏洞。

（5）软件安全开发重视度不够，开发者缺少安全意识。

3. 软件漏洞的类型

漏洞是软件安全威胁的根源，软件漏洞包括设计漏洞和实现漏洞。

（1）设计漏洞：设计错误，往往发现于软件的安全功能特性中。例如，在设计中选用了不当的密码技术，身份鉴别薄弱或缺失，执行有缺陷的输入验证，未做审计日志等。

（2）实现漏洞：来源于软件实际编码中的安全缺陷。包括由编程语言自身不完善引发的安全缺陷（如C++/C语言中一些函数可能引发的缓冲区溢出威胁）；另外程序运行环境的平台实现通常也存在缺陷（如文件系统中用于指向其他文件的符号链接（symbolic link），可被攻击者利用启动系统中的任何程序，从而借此获取权限提升）。

6.1.3 恶意软件威胁

恶意软件（malicious software或malware），是对行为损害系统用户和系统所有者利益的软件的统称，是故意在计算机系统上执行恶意任务的恶意代码的集合。恶意代码是在未被授权的情况下，以破坏软件和硬件设备、窃取用户信息、干扰用户正常使用、扰乱用户心理为目的而编制的软件或代码片段。

2021年7月，国家互联网应急中心发布了《2021年上半年我国互联网网络安全监测数据分析报告》：2021年上半年，捕获计算机恶意程序样本数量约2307万个，计算机恶意程序传播

次数日均达582万余次，涉及恶意程序家族约20.8万个。我国境内感染计算机恶意程序的主机数量约为446万台，同比增长46.8%。位于境外的约4.9万个计算机恶意程序控制服务器控制了我国境内约410万台主机，位于美国、越南和中国香港地区的控制服务器数量分列前三位。

与此同时，随着移动互联网网民数量快速增长，金融服务、生活服务、支付业务等全面向移动互联网应用迁移，包括窃取用户信息、发送垃圾信息、推送广告和欺诈信息等在内的危害移动互联网正常运行的恶意行为在不断侵犯广大移动用户的合法权益。2021年，国家互联网应急中心通过自主捕获和厂商交换获得移动互联网恶意程序86.6万余个，排名前三的分别为流氓行为类、资费消耗类和信息窃取类，占比分别为47.9%、20%和19.2%。

表6.1和表6.2分别给出了2021年瑞星公司根据病毒感染数量、变种数量和代表性等综合评估选出的排名前10的恶意程序和排名前5的手机病毒。

表6.1 2021年恶意程序TOP10

排名	病毒名称	行为特征
1	Adware.Agent!1.C6F0	流氓软件，国内流氓软件使用的流氓模块，主要通过Web下载、共享软件等方式进行传播
2	Trojan.Agent!8.B1E	木马病毒，目的通常为破坏系统、窃取用户隐私、下载其他木马，主要通过电子邮件附件、Web下载等方式进行传播
3	Trojan.ShadowBrokers!.B	方程式小组的黑客工作套件，该工具被病毒广泛用于传播蠕虫病毒，主要通过共享软件、免费软件等方式传播
4	Exploit.UAC!8.107CD	漏洞利用程序，该程序可利用系统漏洞绕过用户账户控制(UAC)，主要通过Web下载、共享软件等方式进行传播
5	Trojan.Zpevdo!8.F912	木马病毒，目的通常为破坏系统、窃取用户隐私、下载其他木马，主要通过电子邮件附件、共享软件等方式进行传播
6	Trojan.Vools!1.B1FA	方程式小组的黑客工作套件，该工具释放并调用永恒之蓝等病毒进行攻击，主要通过漏洞、免费软件、下载站、共享软件等方式进行传播
7	Trojan.Inject!8.103	注入型木马病毒，该病毒会将恶意代码注入进其他程序运行，主要通过Web下载、共享软件等方式进行传播
8	Dropper.Generic!8.35E	释放型木马病毒，该病毒会释放其他具有恶意行为的木马，主要通过电子邮件附件、下载站、共享软件等方式进行传播
9	Worm.Win32.Undef.oa	蠕虫病毒，感染系统文件并自动传播，主要通过受感染文件、漏洞、U盘、共享软件、Web下载等方式进行传播
10	Trojan.Kryptik!8.8	恶意Crypter打包程序，通常用于保护后门、木马及间谍软件，达到逃避安全软件检测目的，主要通过电子邮件附件、Web下载等方式进行传播

表6.2 2021年手机病毒TOP5

排名	病毒名称	行为特征
1	Trojan.SMSreg!8.2DFC	运行后无明显扣费提示，用户若不慎单击会发送扣费短信，造成用户资费损失，主要通过免费软件、下载站、共享软件等方式进行传播
2	Adware.Mobby/Andriod!8.A0FC	广告软件，包含Mobby广告SDK的软件，该软件主要通过Web下载、共享软件等方式进行传播

续表

排名	病毒名称	行为特征
3	Dropper.Agent/Android!8.37E	释放型木马病毒，该病毒会释放其他木马并运行，主要通过下载站、共享软件等方式进行传播
4	Trojan.Obfus/Andriod!8.3F7	带混淆的木马病毒，该病毒常使用混淆工具规避安全软件检测，主要通过免费软件、Web下载等方式进行传播
5	Trojan.Agent/Android!8.358	安卓木马病毒，目的通常为破坏系统、窃取用户隐私、下载其他木马，主要通过免费软件、下载站、共享软件等方式进行传播

6.1.4 软件侵权

软件知识产权是软件开发者对自己的智力劳动成果所依法享有的权利，是一种无形财产。然而，由于计算机软件产品复制成本低、复制效率高，所以往往成为被侵犯版权的对象。

软件版权，又称为计算机软件著作权，是软件开发者或其他权利持有人对其开发的软件作品享有的人身权和财产权，这是法律授予软件著作权的专有权利。人身权包括发表权、开发者身份权；财产权包括使用权、使用许可和获得报酬权、转让权。

计算机软件侵权行为主要有以下几种。

（1）未经软件著作权人的同意而发表或者登记其软件作品。

（2）将他人开发的软件当作自己的作品发表或登记，在他人开发的软件上署名或更改他人开发软件的署名。

（3）未经合作者同意，将与他人合作开发的软件当作自己独立完成的作品发表或登记。

（4）未经软件著作权人或其合法受让者的许可，修改、翻译、复制或部分复制其软件作品。

（5）未经软件著作权人及其合法受让者同意，向公众发行、出租其软件的复制品，或是向第三方办理软件权利许可或转让事宜，以及通过信息网络传播著作权人的软件。

软件的开发需要投入大量的精力和财力，软件本身是高度智慧的结晶，与有形财产一样，也应受到法律的保护，以提高开发者的积极性和创造性，促进软件产业的发展。打击侵权盗版，保护软件知识产权，建立一个尊重知识、尊重知识产权的良好市场环境是政府的意向，也是软件企业的愿望，它将关系到软件产业的发展和软件企业的存亡。

在软件侵权行为中，对于一些侵权主体比较明确的，一般通过法律手段予以解决，但是对于一些侵权主体比较隐蔽或分散的，政府管理部门受时间、人力和财力诸多因素的制约，还不能进行全面管制，因此有必要通过技术手段来保护软件不被侵权。

6.2 软件安全开发

软件安全的问题是软件自身的缺陷问题，主要在软件设计和软件实现的过程中产生，具体表现在软件设计的架构问题和实现上的错误。如果把软件开发比作建造房子，那么实现上的软件代码错误相当于墙没起好，而架构问题引发的风险则相当于房梁有问题。由此可见，在软件安全问题上，架构上的风险往往比实现上的错误更重要、更难理解。

在软件和信息系统的开发过程中，由于技术难度高、项目复杂、开发周期短而带来的一系列困难，潜伏安全性隐患的概率其实是很大的。软件的安全性并不能通过传统的软件开发管理流程得到实现，需要在软件开发生命周期各阶段采取必要的、相适应的安全措施来避免绝大

多数的安全漏洞。

6.2.1 传统软件开发中的安全局限性

为解决软件危机，人们从20世纪70年代开始探索用工程化的方法进行软件开发，软件开发工程化的概念和方法应运而生，由此诞生了软件工程学。它以“工程化”的思想来开发与维护软件，从时间角度对软件开发和维护的复杂问题进行分解，把软件生命的漫长周期依次划分为若干阶段，每个阶段有相对独立的任务。

软件从定义开始，经过开发、使用和维护，直到最终退役的全过程为软件生命周期。软件生命周期通常划分为软件定义、软件开发和软件运行维护等时期，每个时期又可以进一步划分成若干阶段，如图6.2所示。

然而进入21世纪后，软件发展趋势之一是功能越来越多、越来越复杂，使得软件开发暴露出诸多安全问题。例如，软件开发周期短、工作量大、无暇顾及安全，软件设计时缺乏安全设计考虑，软件开发人员缺乏安全编程经验，还要应对互联网环境下的安全挑战等。统计数据显示，即使采用严格的软件开发质量管理机制，普通的软件工程师开发的每千行代码中还是存在20个缺陷，普通软件开发公司的缺陷密度（每千行代码存在的缺陷数）为4～40个缺陷。传统软件开发的局限性体现在以下三方面。

（1）传统软件开发教育局限性。软件开发通常着重于软件工程、数据结构、编译原理、系统结构、程序语言等理论及实践的教育，但是安全开发教育常常被忽视，缺乏有效的安全开发教育，不利于开发人员安全意识和习惯的养成。

（2）传统开发人员局限性。由于安全开发教育的缺乏，使得开发人员对安全问题没有足够理解，不了解安全设计的基本原理，不知道安全漏洞的常见类型，也不知道如何设计针对安全的测试数据，在整个软件开发生命周期各环节都缺少必要的对安全措施的考量和实施。

（3）传统软件生命周期局限性。由图6.2可知，软件生命周期包括需求分析、总体设计、编码与单元测试和软件维护等阶段，但是明显缺少安全介入的阶段。实际上，在软件开发生命周期各阶段采取必要的、相适应的安全措施可以避免绝大多数的安全漏洞。

6.2.2 安全软件开发生命周期

软件安全开发研究始于20世纪末，主要研究如何采取措施防止由于设计、开发、提交、升级或维护过程中的缺陷而导致的系统脆弱性。在传统软件生命周期的基础上，定义了安全软件开发生命周期（Secure Software Development Lifecycle，SSDL），旨在通过软件开发的各个步骤来确保软件的安全性，其目标是确保安全的软件得以成功实现。通常由以下5个主要部分组成。

1. 安全原则、规则和规章

安全原则、规则和规章通常被视为保护性需求。该阶段应创建一份系统范围内的规范，其中定义将应用到本系统的安全需求，此规范也可以通过特定的官方规章来定义，如（Open Web Application Security Project，OWASP）的Web应用程序安全标准（Web Application Security Scanner，WASS）、支付卡行业数据安全标准（the graham-leech-bliley）等。若软件系统需要遵循这些特定行业的安全标准，在需求分析时就要考虑到这些要求。若某些系统不受任何规则条例的影响，仍然应该开发一个安全策略，这些安全策略要以文档的形式记录下来，并通过对其跟踪和评估，使其成为一个不断发展的基本规则。

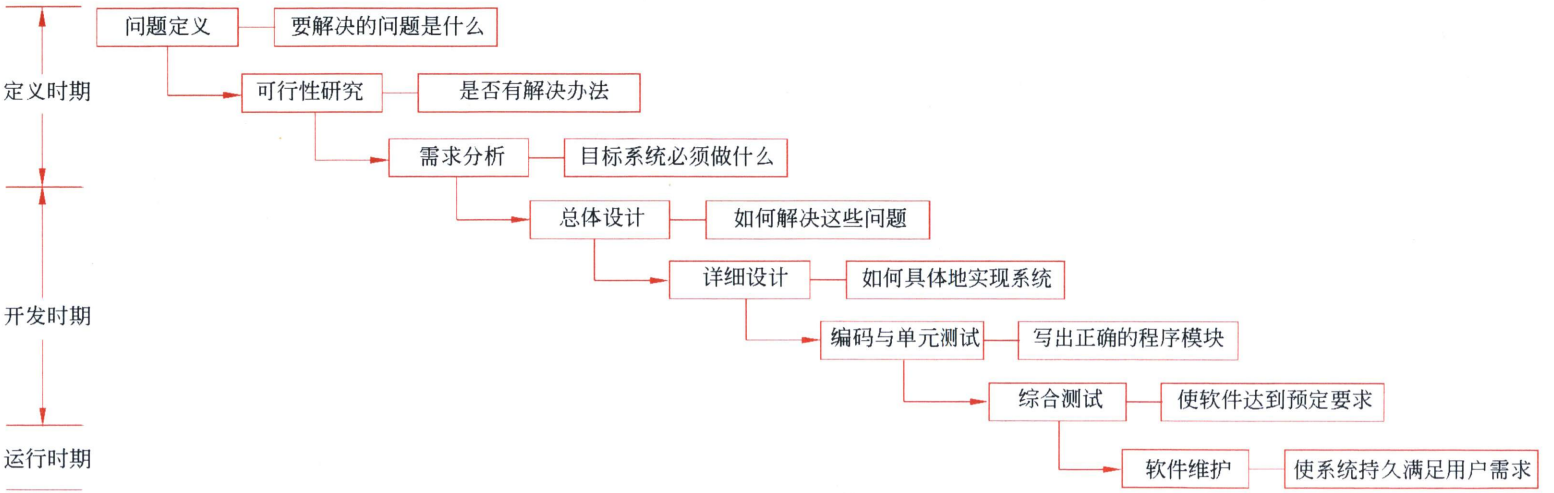


图 6.2　软件生命周期各阶段及任务

2. 安全需求工程

这里通常指特定功能需求所需的特有安全需求，这些安全需求有别于系统范围的安全要略和安全规范，这种安全需求工程通过文档在项目起始的时候定义不许系统以哪种传统的需求分析（主要是功能角度分析需要哪些功能），安全需求则是定义不许系统以哪种方式处理某功能。分析人员通常是从攻击者的角度看待系统应注意的地方，可以通过开发“滥用用例”来展现不允许和未授权的动作流，以及可能被攻击的方式。用例的包含关系可以阐述许多保护机制，如登录过程；用例的扩展关系可以阐明许多检测机制，如审计日志。需求一般包括：缺点、错误预防点，即定义了应该避免的缺点和错误；安全需求处理点的关联，即在多个地方对安全需求进行了处理，这些地方可以关联在一起。

3. 架构和设计评审、威胁建模

软件的架构和设计应该被安全分析人员尽早评审，避免形成有安全缺陷的体系结构和设计。为了避免设计漏洞，在软件系统分析和设计阶段就应考虑可能面临的安全威胁，需要进行威胁建模（例如，系统是否需要实体认证，是否需要保护信息的私密性），这有利于及早发现安全问题。

4. 软件安全编码

代码的实现者要对软件漏洞的来源有所了解，软件编码人员应该遵照一些软件编码原则，如不使用 strcpy 而使用 strncpy 等。静态源代码分析工具可以自动发现一些潜在的源代码安全缺陷，并加以警告。二进制代码审查工具也能够帮助编码人员发现一些第三方调用库中的安全问题，以提高软件的整体安全性。

5. 软件安全测试

软件安全测试包括白箱测试、黑箱测试、软件渗透测试、基本风险的测试，可以判定漏洞的可利用性，即对测试出的安全漏洞或在开发结束新公布的软件漏洞进行分析，判定这些漏洞是否可被攻击者利用，构成威胁。

其中需要考虑的方面还包括软件安全发布、部署与维护。具体包括：软件代码的保护、版权保护和反盗版，软件安全用户权限，补丁管理，软件安全升级。

无论采用何种软件开发周期模型，安全都应该与其紧密结合，将传统软件开发生命周期和安全软件开发生命周期两者间关系进行关联和结合。图 6.3 展示了安全软件开发生命周期（SSDL）与传统软件开发生命周期的关系，这里传统软件开发生命周期在 SSDL 的外面一层呈现。

6.2.3 其他安全软件开发生命周期模型

1. 微软可信计算安全开发生命周期

为提高微软软件产品的安全性，微软公司提出了可信计算安全开发生命周期（trustworthy computing security development lifecycle），简称为安全开发生命周期（Security Development Lifecycle，SDL）。2004 年以来，微软一直将 SDL 作为全公司的强制性策略，用于减少软件中安全漏洞的数量和严重性。Windows Vista 是微软第一个采用 SDL 过程开发的操作系统。

SDL 从安全角度指导软件开发过程，包括威胁建模、静态分析和安全审查等安全技术，确保减少微软产品的安全漏洞，保证最终用户的信息安全。在微软的软件开发各个阶段，SDL 添加了安全活动和业务活动目标。SDL 将软件开发生命周期划分为 7 个阶段，提出了 17 项

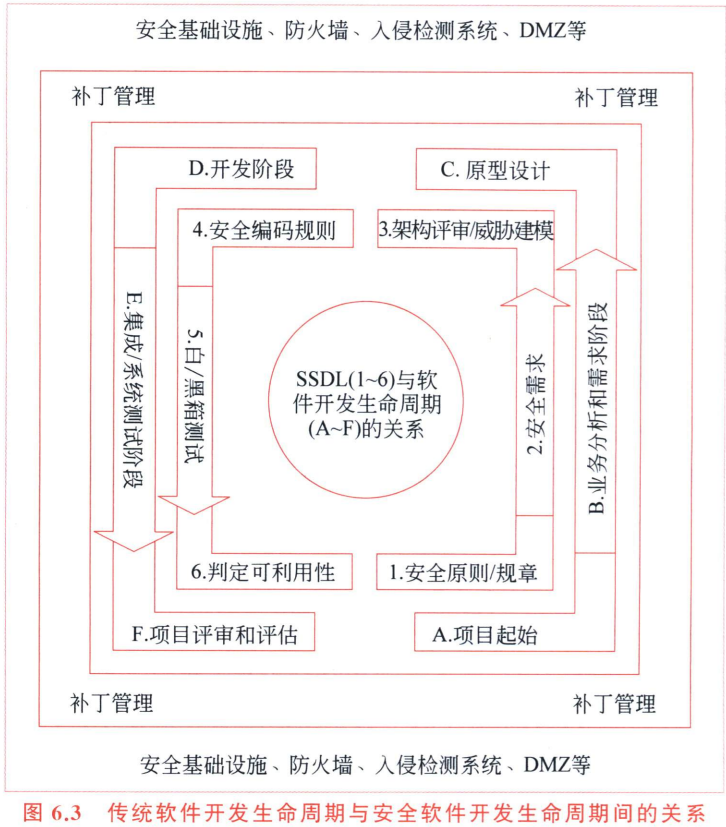


图 6.3 传统软件开发生命周期与安全软件开发生命周期间的关系

重要的安全活动，如图 6.4 所示。

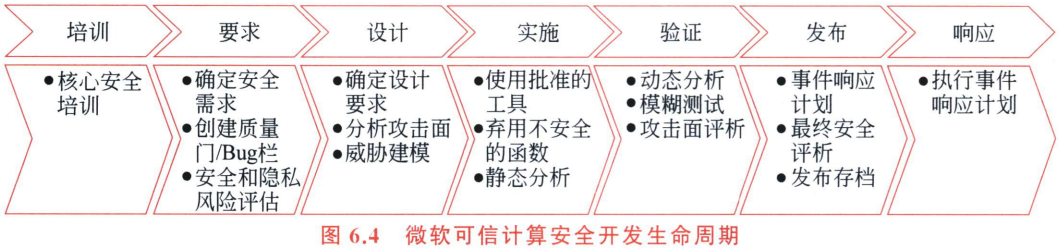


图 6.4 微软可信计算安全开发生命周期

SDL 各阶段的主要任务和目的如下。

（1）培训：在软件开发初始阶段，对开发团队和高层管理者进行安全意识与安全开发能力培训。确保开发人员有足够的能力完成软件安全开发的任务；确保管理者对安全开发足够重视，提供必要的支持。

（2）需求：在软件需求分析阶段，开发团队与高层管理者确定软件需要遵循的安全标准和相关要求，分析软件的安全需求，建立安全和隐私要求的最低可接受级别。

（3）设计：在软件设计阶段，根据开发的安全需求，从安全性的角度定义软件的总体结构。通过分析攻击面，设计相应的功能和策略，规避或减少不必要的安全风险，同时通过威胁建模，分析软件或系统的安全威胁，提出缓解措施。

（4）实施：在软件实现阶段，按照安全设计要求，对软件进行编码和集成，实现满足安全设计要求的功能、策略及缓解措施。在该阶段通过遵循安全编码规则和禁用不安全的 API，减少实现时导致的安全问题和由编码引入的安全漏洞，并通过代码静态分析等措施来确保安全

编码规范的实施。

(5) 验证：在软件测试阶段，通过动态分析和安全性测试，全面核查攻击面，检测软件是否存在安全漏洞，检查各关键因素上的威胁缓解措施是否得以正确实现。

(6) 发布：在产品发布阶段，建立可持续的安全维护和响应计划，对软件进行最终安全核查。本阶段应将所有相关信息和数据存档，建立并实施移交计划，以便对软件进行发布与维护。这些信息和数据包括所有规范、源代码、二进制文件、专用符号、威胁模型、文档、应急响应计划、任何第三方软件的许可证和服务条款，以及执行发布后维护任务所需的任何其他数据。

(7) 响应：在产品交付用户使用后，及时响应安全事件与用户提交的漏洞报告，实施漏洞修复和应急响应。同时对发现的问题，尤其是安全问题进行分析，并将分析结果用于 SDL 的持续改进。

在这 7 个阶段中，前 6 个阶段的 16 项安全活动是开发团队必须成功完成的，活动是否成功由安全专家确认，并且会作为评估过程的一部分，不断进行有效性评判。微软公司的实践经验表明，利用 SDL 来开发产品的安全性，将安全活动作为软件开发过程的一部分来执行，其安全效益大于零散或临时实施的活动。

2. 安全软件开发之团队软件过程

安全软件开发之团队软件过程(Team Software Process for Secure software development，TSP-Secure)是由卡内基梅隆大学软件工程学院基于团队软件过程(Team Software Process，TSP)提出的。TSP 为开发软件产品的开发团队提供指导，其早期实践侧重于帮助开发团队改善软件质量和生产率，以更好满足开发成本及进度目标。TSP 为适用于团体和个人的软件工程提供了一个框架，通过 TSP 产生的软件比起现有方法产生的软件要少一个或者两个数量级的缺陷数目。

TSP-Secure 将 TSP 进一步扩展，它直接专注于软件应用安全，目标是在软件设计和实施过程中减少或消除漏洞，提供评估和预测其他软件开发商所交付的软件中可能存在漏洞情况的能力。为此，TSP-Secure 定义了一套开发安全软件的过程和方法，从以下 3 方面陈述了安全软件开发。

(1) 考虑到安全软件不是偶然建立的，TSP-Secure 陈述了安全计划，帮助建立自我导向的开发团队。

(2) 由于软件安全和质量紧密相关，TSP-Secure 在整个产品的开发生命周期中帮助管理质量。

(3) 考虑到开发团队具备软件安全意识的重要性，TSP-Secure 包括对开发人员安全意识的训练。

TSP-Secure 质量管理的策略是在软件开发生命周期中，去除多个缺陷点。去除的缺陷点越多，在提出它们之后立刻找到问题的可能性越大，这使得问题能够被轻松地修复。

每个可除缺陷的活动可以被认为是一个过滤器，用于删除一定百分比的可能导致软件产品漏洞的缺陷，如图 6.5 所示。在软件开发生命周期中，去除缺陷的过滤器越多，那么在产品发布时，软件产品中剩余的可能导致软件漏洞的缺陷则会越少。更重要的是，早期测量到缺陷，能够使组织在软件开发生命周期的早期采取纠正措施。

每当一个缺陷被移除时，安全性被重新度量，每个缺陷移除点也将变成度量点。这种度量甚至比缺陷移除和防止还重要，因为它可以告诉一个团队，他们现在的状况，帮助他们决定是要移动到下一个步骤，还是停止并采取纠正措施，并且指示他们，为了达到目标，应该在哪些位

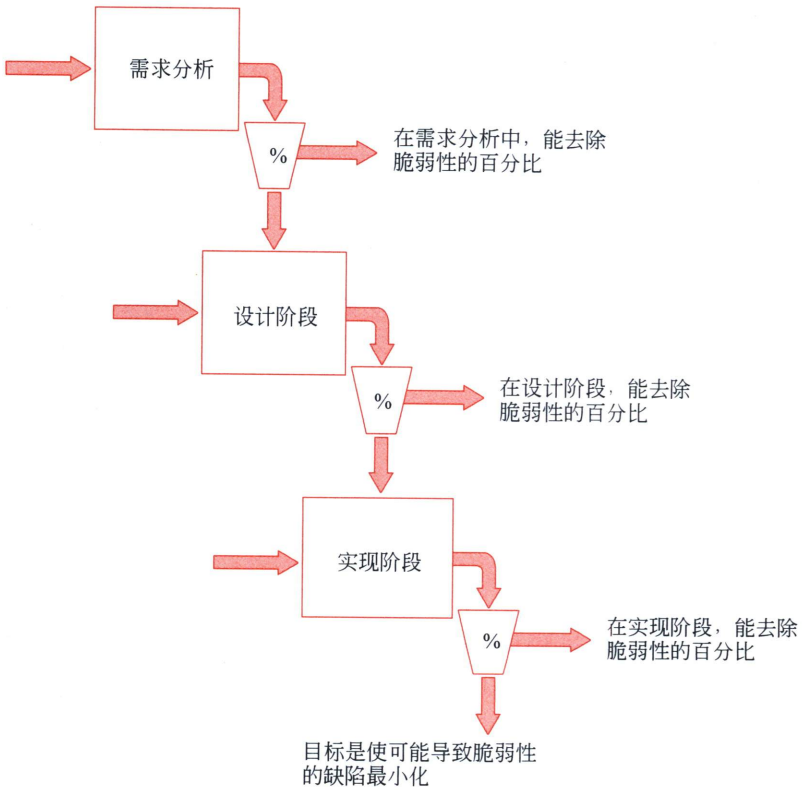


图 6.5 去除漏洞过滤器

置修复进程。

3. BSI 成熟度模型

BSI 成熟度模型(Building Security In Maturity Model，BSIMM)是一个针对多个软件公司的软件安全项目进行研究的模型，该模型对不同软件安全项目所采取的措施和实践进行量化，描述那些共有的基础活动特征及那些具有唯一特征的变化，即共性和各自的特点。

很多软件公司一方面希望能构建功能强大而又安全的软件，另一方面却不知道该如何构建，没有其他公司的最佳实践作为参考。针对此问题，BSIMM 分析了多个真实的软件安全项目的实践数据，提炼出这些软件公司和软件项目中的共同特点，其目的是帮助更大范围的软件安全公司对其各自的项目进行安全规划、实现和评测。因为 BSIMM 中分析的原始数据来自于当前国际上很多著名的大型公司，包括美国银行、美国第一资本银行，微软、谷歌、英特尔及赛门铁克等公司，所以其分析结果体现了当今业界软件安全技术的领先水平。而对软件安全感兴趣的公司，可以参考这个结果来指导自己的软件安全开发实践。

因此，BSIMM 是用来帮助软件公司理解和设计安全的软件的。如果使用得当，BSIMM 可以帮助确定公司对于现实世界的软件安全要求间的差距，以及可以采取哪些步骤使方法更有效。BSIMM 的目标是构建和不断发展软件安全行动的指南。通过明确指出的目标和目的，并通过根据度量跟踪适合自身的做法，可以有条不紊地将软件安全建立到公司的软件开发实践中来。通过开展 BSIMM 所述的活动，可以逐步发展安全计划，在最佳的时间里，实现高水平的软件安全，而不需要过度的开销。

由于 BSIMM 是基于各个公司的具体实践，它可以被看作一种事实的标准。它提供了判别某做法是否是通常被采纳的实践做法的真实而有说服力的依据。而且，不像很多官方的标

准，它认为并非所有的公司都需要达到相同的安全性目标。没有一个公司会需要执行所有的行为，这个模型确实提供了一个潜在的度量所有公司的基准，并演示了其流程。

6.2.4 安全编码策略和原则

传统的软件开发组织常常把软件的功能、任务时间表和开发成本放在关注的首位，而把软件的安全和质量放在其次，这已经无法适应现代软件工程的需求。因此，在编码过程中，必须考虑安全性问题。

健全的编码可以大大减少软件实现期间引入的漏洞，在编码过程中需要遵循安全编码策略。目前国际标准化组织(ISO)、计算机安全应急响应组(CERT)等提出的安全编程国际标准(如CERT维护针对C、C++、Java、Perl四种编程语言的安全编程指南)，可以为开发者提供指导和建议，帮助开发健壮、安全的软件。国际标准中涉及的安全编程规范可以概括为如下的通用安全编码准则。

(1) 验证输入。安全程序第一道防线是检查每一个不可信的输入。如果能保证不让恶意的数据进入程序，或者至少不在程序中处理，那么程序在面对攻击时将更加健壮。这与防火墙保护计算机的原理类似：它不能预防所有的攻击，但它可以让一个程序更加稳定。这个过程称为验证(检查或过滤)输入。那么，应该在何处进行验证呢？通常，在数据最初进入程序时，或者是在一个低层次的例程实际使用这些数据时进行验证；这样，即使攻击者成功地突破了一道防线，也还会遇到另一道。其中，最重要的规则是所有的数据必须在使用之前被验证。

(2) 避免缓冲区溢出。缓冲区溢出是一个非常普遍而且严重的问题。一旦发生溢出，轻则使远程服务程序或本地程序崩溃，重则攻击者可以利用溢出执行任意代码。C/C++语言特性决定了用其编写的程序容易发生溢出。比如，对于定义好的int buff[10]，只有buff[0]-buff[9]的空间是定义时申请的合法空间，如果往里面写入数据时出现buff[12]=010，那么就越界了，会引发溢出。C语言常用的strcpy()、sprintf()、strcat()等函数都非常容易导致缓冲区溢出问题。应对缓冲区溢出常见的也是最重要的方法主要有：正确的编写代码，使用没有缓冲区溢出问题的函数，填充数据时计算边界，程序指针完整性检查，非执行的堆栈防御等。

(3) 错误和异常处理。程序在运行过程中可能会出现错误而中断正常的控制流，这就是异常现象。异常的代码的程序可能会在不期望终止的时候终止，甚至可能引发严重问题。例如，如果程序试图从存款账户上把钱转移到支票账户上，但是因为运行时错误，在把钱从账户上提取之后和把钱存入支票之前，程序被终止，那么用户将会有经济损失。处理程序错误和异常可从以下几方面着手。

- 检查程序内部接口数据，保证程序内部接口安全。
- “安全地”失败：检测到异常，安全处理各种可能运行路径；检测到某些错误行为或是数据，必须以合适的方式处理，保证程序运行安全；必要时立即拒绝服务，甚至不回送详细的错误代码。
- 最小化反馈：无论程序运行成功或失败，避免给予不可靠用户过多的信息；认证程序在认证前也要尽量少提供信息；不要返回程序接受的密码。
- 避免拒绝服务攻击：输入错误尽快返回，并设置超时时限及延时服务来缓解攻击影响。
- 避免竞争条件：使用原子操作或锁操作适当控制对共享资源(如文件/变量)等的访问，避免死锁等情况发生。

- 安全使用临时文件：避免引入通过访问已知文件名或可猜测的临时文件的安全漏洞。

（4）安全调用其他组件。应用程序通常都会调用其他组件，如底层的操作系统、数据库、可重用的库、网络服务等。在调用组件时，确保使用安全组件，并且只采用以下安全的方式使用。

- 检查组件文档，搜索相关说明，确保组件安全。
- 尽量使用经过认可的组件。
- 尽可能不调用外部命令，如果不得已要调用，则必须严格检查参数。
- 正确处理返回值，无论调用成功或失败，都要检查返回值。调用成功时检查返回值是否按照期望值处理，数据中是否含有 NUL 字符、无效字符或其他可能产生问题的东西；调用发生错误时则需检查错误码。
- 保护应用程序和组件之间传递的数据，具体视安全需求和安全环境，如考虑传输加密，则要采用密码算法和安全协议。

（5）程序编写和编译。编写程序时，要减少潜在可被利用的编码结构或设计；注意及时清除密码和密钥等敏感数据；确保程序只实现指定的功能，并且永远不要信任用户输入。编译程序时，尽可能使用最新版本编译器与支持工具，并且用好编译器内置防御特性。

软件安全是极其复杂的问题，其复杂性一直在不断突破人数能应对的极限，想要对所有问题进行设计和测试是不现实的，出现故障也不可避免。因此，除在设计和开发过程中须遵循安全编码准则外，还可辅以源代码审核，检测并报告源代码中可能导致安全漏洞的薄弱之处。

源代码审核关注编码中的实现缺陷，通常通过静态分析工具进行，用这些工具扫描源代码，能够发现大约 50% 的安全问题。目前源代码审核工具包括 Coverity、Fortify、Ounce Labs、SecureSoftware 等商业工具，也有如 BOON、Cqual、Xg++、FindBugs 等免费或开源工具，利用这些工具可自动快速地进行代码检查，相比于费时费力、容易遗漏的人工审核，具有明显优势。

6.3 恶意代码分析

对计算机系统来说，最复杂的威胁可能就是那些利用计算机系统弱点来进行攻击的恶意代码。恶意代码是在未被授权的情况下，以破坏软件和硬件设备、窃取用户信息、干扰用户正常使用、扰乱用户心理为目的而编制的软件或代码片段。其实现方式可以有多种，如二进制执行代码、脚本语言代码、宏代码或是寄生在其他代码或启动扇区中的一段指令。

恶意代码已经成为攻击计算机信息系统主要的载体，其攻击的威力越来越大，攻击的范围越来越广，是软件安全的主要威胁之一。本节将介绍恶意代码的分类、原理及其检测与防护技术。

6.3.1 恶意代码分类及其区别

恶意代码种类很多，图 6.6 给出了从对主机程序的依赖性及传染性划分恶意代码的一种分类方式。

从图 6.6 可以看出，从对主机依赖的角度可以将恶意代码分为依赖主机程序的恶意软件和独立于主机程序的恶意软件。进一步地，还可以根据是否具有传染性将依赖和独立于主机程序的恶意代码细分为传染的依赖性恶意代码、不可传染的依赖性恶意代码、传染的独立性恶

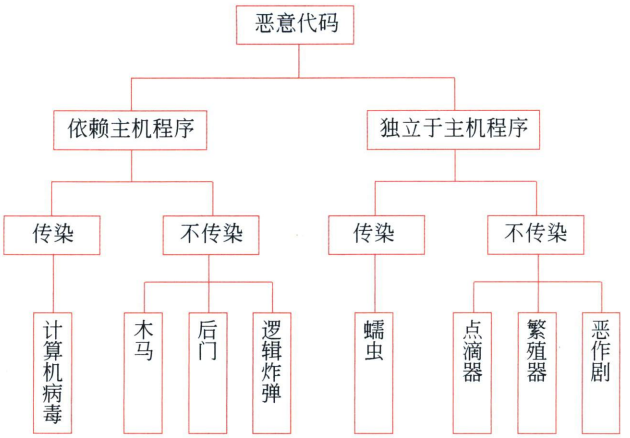


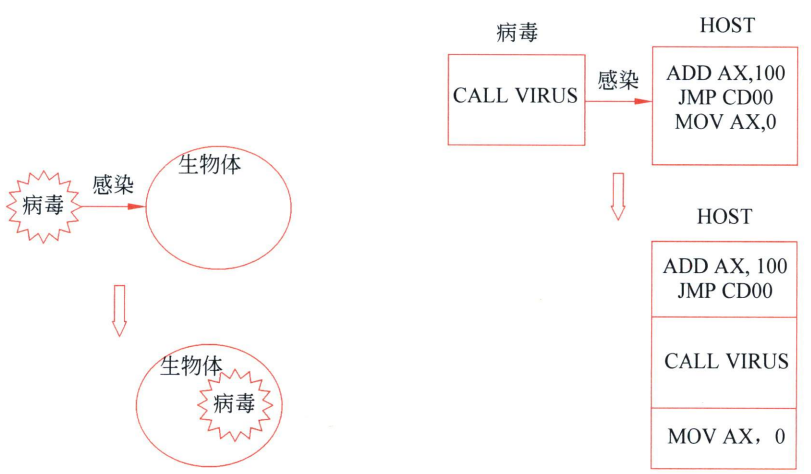
图 6.6 恶意代码分类

意代码和不传染的独立性恶意代码。

1. 计算机病毒

计算机病毒是一种人为制造的、能够进行自我复制的、对计算机资源具有破坏作用的一组程序或指令的集合。1994 年 2 月 18 日公布的《中华人民共和国计算机信息系统安全保护条例》中对计算机病毒的定义为："计算机病毒是指编制或者在计算机程序中插入的破坏计算机功能或者破坏数据，影响计算机使用并且能够自我复制的一组计算机指令或程序代码。"

计算机病毒与生物病毒一样，有其自身的病毒体（病毒程序）和寄生体（宿主（host），病毒载体）。寄生体为病毒提供一种生存环境，是一种合法程序。当病毒程序寄生于合法程序后，成为程序的一部分；并随着合法程序的执行而执行，也随着合法程序的消失而消失。病毒感染示意如图 6.7 所示。



(a) 生物病毒感染与寄生　　(b) 计算机病毒感染与寄生

图 6.7 病毒感染示意

由图 6.7(b)可以看出，计算机病毒将宿主作为攻击对象并感染之。为了增强活力，病毒程序通常寄生于一个或多个被频繁调用的宿主程序中。目标宿主对象的数量和类型随病毒的不同而有所区别，下面给出最常见的宿主对象示例。

（1）可执行文件。通过其自身附加到宿主程序进行复制的典型病毒类型的目标对象。除了使用.exe扩展名的典型可执行文件之外，具有扩展名.com、.sys、.dll等文件也可用于此用途。

（2）脚本。脚本病毒通常将脚本程序作为宿主对象，包括使用如Microsoft Visual Basic Script、Java Script、Apple Script之类的脚本语言。此类文件的扩展名包括.vbs、.js和.prl。

（3）宏。宏病毒的目标宿主是宏脚本语言的文件。例如，病毒可在Microsoft Word中使用宏语言来生成许多效果，包括恶作剧效果（在文档中改变单词或更改颜色）、恶意攻击效果（格式化计算机的硬盘驱动器）。

（4）启动扇区。计算机磁盘上的特定区域（如主启动目录记录MBR）也可以作为病毒宿主，因为它可以执行恶意代码。当某个磁盘被感染，如果使用该磁盘来启动其他计算机系统，将会复制、传播病毒。

计算机病毒种类繁多、特征各异，但一般具有以下特性：自我复制能力；很强的感染性；一定的潜伏性；特定的触发性；很大的破坏性。其中，自我复制是病毒代码的明显特征。感染计算机病毒后，计算机在每次运行正常程序之前，均先运行一次病毒程序，将病毒循环往复地读入内存或执行病毒指令，从而使病毒不断被复制，并在系统中得以传播。另外，破坏性也是计算机病毒的主要特征，因为病毒在自我复制时可能会损坏数据、消耗资源并占用网络带宽。

2. 蠕虫

蠕虫（worm）是一种独立的可执行程序，它通过将自己复制到宿主计算机上，然后利用此计算机的通信信道进行复制。与计算机病毒不同，蠕虫并不需要将自身插入宿主程序来达到自我复制的目的。通常，蠕虫传播无须用户操作，并可通过网络分发它自己的完整副本（可能有改动）。

蠕虫的工作流程可以分为漏洞扫描、攻击、传染、现场处理四个阶段，如图6.8所示。蠕虫扫描到有漏洞的计算机系统后，将蠕虫主体迁移到目标主机；然后，蠕虫进入被感染的系统，对目标主机进行现场处理。现场处理部分的工作包括隐藏保护、信息搜集等。同时，蠕虫生成多个副本，重复上述流程，从而把自身的备份带入其他未被感染的计算机系统。

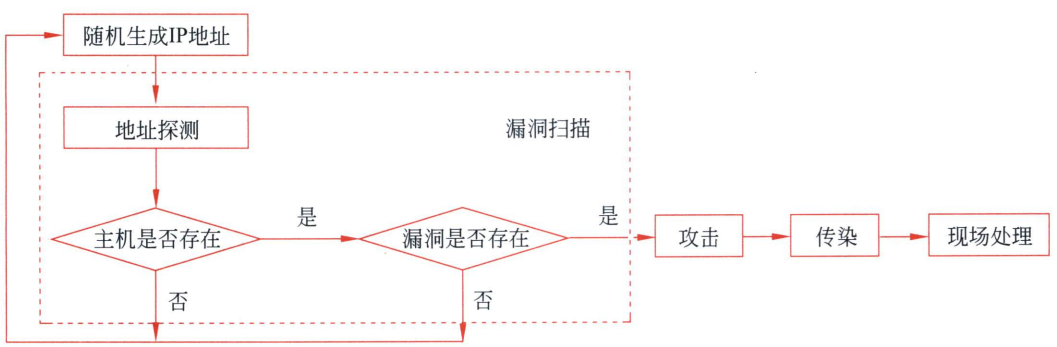


图6.8 蠕虫的工作流程

蠕虫和计算机病毒都具有传染性和复制功能，这两个主要特性上的一致，导致二者难以区别，尤其是近年来，越来越多的病毒采取了部分蠕虫的技术。同时，具有破坏性的蠕虫也采取了部分病毒的技术。但对计算机病毒与蠕虫进行区分还是非常必要的，因为通过对它们之间的区别、不同功能特性的分析，才可以确定谁是对抗蠕虫的主要因素，谁是对抗计算机病毒的主要因素，可以找出有针对性的有效对抗方案，同时也为对它们的进一步研究奠定理论基础。蠕虫与病毒的区别如表6.3所示。

表 6.3 蠕虫与病毒的区别

属 性	病 毒	蠕 虫
存在形式	寄生	独立个体
复制机制	插入宿主程序(文件)中	自身的复制
传染机制	宿主程序运行	系统存在漏洞
搜索机制	主要针对本地文件	主要针对网络上的计算机
触发传染	计算机使用者	程序自身
影响重点	文件系统	网络性能,系统性能
计算机使用者角色	病毒传播中的关键环节	无关
防治措施	从宿主程序中摘除	为系统打补丁
对抗主体	计算机使用者、反病毒厂商	系统提供商,网络管理员

由表 6.3 可知,病毒主要攻击文件系统,传染过程中,计算机使用者是传染的触发者,计算机使用者的水平高低常常决定了病毒所能造成破坏的程度;蠕虫主要利用计算机系统漏洞传染,搜索到存在漏洞的计算机后主动攻击,与计算机使用者是否进行操作无关,从而与使用者的计算机知识水平无关。另外,蠕虫强调自身副本的完整性和独立性,这也是区分蠕虫和病毒的重要因素,可以通过简单地观察攻击程序是否存在载体来区分蠕虫和病毒。

蠕虫的最大特点是利用各种安全漏洞进行自动传播。在网络环境下,蠕虫可以按指数增长模式进行传染。蠕虫侵入计算机网络,可以导致计算机网络效率急剧下降、系统资源遭到严重破坏,短时间内造成网络系统瘫痪。

3. 木马

木马(trojan horse)是一种在远程计算机之间建立连接、使远程计算机能通过网络控制本地计算机的非法程序。木马的名称源于古希腊神话特洛伊木马记,它是一种恶意程序,是一种基于远程控制的黑客工具,一旦侵入用户计算机,就悄悄地在宿主计算机上运行,在用户毫无察觉的情况下,让攻击者获得远程访问和控制系统的权限,进而在用户计算机中修改文件、修改注册表、控制鼠标、监视/控制键盘或窃取用户信息。它是攻击者的主要攻击手段之一,具有隐蔽性和非授权性等特点。

多数木马包括客户端和服务器端两部分,即采用服务器/客户端结构,功能上由木马配置程序、控制程序和服务器程序 3 部分组成,如图 6.9 所示。

木马服务器程序驻留在受害者的系统中,非法获取其操作权限,负责接收控制指令,并根据指令或配置发送数据给控制端。木马配置程序设置木马程序的端口号、触发条件、木马名称等,使其在服务器端隐藏得更隐蔽。木马控制程序控制远程木马服务器(有些控制程序集成了木马的配置功能),统称为控制端(客户端)程序,负责配置服务器、给服务器发送指令,同时接收服务器传过来的数据。

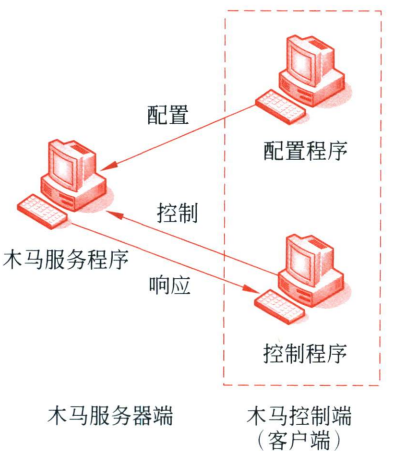


图 6.9 木马的结构

攻击者通常利用一种称为绑定程序的工具将木马服

务器绑定到某个合法软件上。只要用户运行该软件，木马的服务器能在用户毫无察觉的情况下完成安装过程。控制端和服务器端都在线的情况下，控制端可以通过木马端口与服务器端建立连接。木马连接建立后，控制端端口和木马服务器端之间将会出现一条通信通道，控制端程序可以通过这条通道及木马服务器程序对服务器端进行远程控制。

攻击者经常利用木马入侵大量的计算机（被控制的机器有时候被称为“僵尸”“肉鸡”），用广播方式发布命令，指示所有在其控制下的计算机一起行动，或者向更广泛的范围传播木马，或者针对某一要害主机发起分布式拒绝服务攻击。

木马可以通过一般的计算机病毒传播，还可以利用系统的漏洞进行植入，与蠕虫结合起来。但是相对于传统病毒和蠕虫，木马有很多不同表现。病毒的定义强调自我复制的传染性特点，木马的名称强调意图和功能。木马一般不进行自我复制，但具有寄生性，如捆绑在合法程序中得到安装、启动木马的权限，DLL 木马甚至采用动态嵌入技术寄生在合法程序的进程中。木马的最终意图是窃取信息、实施远程监控。木马与合法远程控制软件的主要区别在于是否具有隐蔽性、是否具有非授权性。蠕虫和木马的共性是自我传播，都不感染其他文件。它们的区别是木马需要诱骗用户上当后进行传播；而蠕虫包含自我复制程序，它利用所在的系统进行传播。蠕虫的破坏性更多地体现在耗费系统资源拒绝服务攻击上，木马除助力发动拒绝服务攻击外，还体现在秘密窃取用户信息上。

4. 后门

后门（backdoor 或 trapdoor）是一种可以绕过安全控制而获取对程序或系统访问权的方法。后门实质上是一个模块的、秘密的、未记入文档的入口，一个访问系统或控制系统的通道。后门的产生有三种情况：一是软件厂商或开发者留下的，二是软件设置或编程漏洞造成的，三是攻击者植入的。

（1）厂商或开发者用于软件开发调试留下的后门：在软件开发与调试期间，开发人员常常为了测试一个模块；或者为了今后的修改与扩充；又或者为了在程序正式运行后，当程序发生故障时能够访问系统内部信息等目的而有意识预留后门。这种后门可以被开发者用于上述正常目的，也可以被攻击者用于非正当手段。例如，恶意攻击者把后门当作远程连接系统的工具，在运行后门的主机上打开一个网络端口，然后侦听的后门程序会等待攻击者的远程连接，其通常会和木马功能混合使用。

（2）软件设计或编程漏洞产生的后门：软件设计和编码可能存在漏洞，某些后门就利用了程序的设计缺陷。有些应用程序，如 SMTP 的早期实现具有允许执行某一命令（如调试命令 debug）的功能，著名的 Morris 蠕虫就是使用这个命令在远程执行它自己，如果系统安装了这一有后门的程序，蠕虫就会通过将此命令放置在邮件收件人的位置来实现。

不论是开发者有意还是无意留下的后门，如果在软件开发结束后不及时删除后门，后门就可能被软件的开发者秘密使用，也可能被攻击者发现并利用而成为安全隐患。

（3）攻击者在软件中植入的后门：后门也可能是恶意的软件开发者故意放置在软件中的，还可能是攻击者为了自己能够顺利重返被入侵系统而设置的。

无论是上述哪一种情况，后门都仅仅是一个访问系统或控制系统的通道，其本身并不具有其他恶意代码的直接攻击行为。因此，后门和计算机病毒、蠕虫的最大差别在于，后门不会感染其他计算机。后门与木马的相似之处在于，它们都隐藏在用户系统中，本身具有一定的权限，以便远程机器对本机的控制。它们的区别在于，木马通常是一个完整的软件，而后门是系统中软件所具有的特定功能。

5. 勒索软件

勒索软件(ransomware)是一种极具传播性、破坏性的恶意软件，黑客用来劫持用户资产或资源，使用户数据资产或计算资源无法正常使用，并以此为条件向用户勒索钱财。这类用户资产包括文档、邮件、数据库、源代码、图像、压缩文件等。勒索软件通常会将这些文件进行某种形式的加密操作，使之不可用，或者通过修改系统配置文件，干扰用户正常使用系统的方法，使系统可用性降低，然后通过弹出窗口、对话框或生成文本文件等方式向用户发出勒索通知，要求用户支付赎金来获得解密文件的密码或获得恢复系统运行的方法。

勒索软件攻击的典型流程分为探测侦查、攻击入侵、病毒植入、实施勒索4个阶段。

(1) 探测侦查阶段：收集用户计算机基础信息、找到攻击入口并建立内部立足点。

(2) 攻击入侵阶段：部署攻击资源、侦查网络资产并提升访问权限。

(3) 病毒植入阶段：植入勒索软件、破坏检测防御机制并扩展感染范围。

(4) 实施勒索阶段：窃取机密数据、加密关键数据后加载勒索信息。

一旦攻击得手，攻击者会加载勒索信息，威胁攻击目标支付勒索赎金。通常勒索信息包括通过暗网论坛与攻击者联系的联系方式、以加密货币支付赎金的支付方式、支付赎金后获取解密工具的方式、警告受害者不要自行解密或修改数据名称等。图6.10所示即为著名的WannaCry勒索软件界面，勒索软件的赎金形式除比特币外，也常见真实货币或其他虚拟货币。

图6.10 WannaCry勒索软件界面

勒索软件的传播手段与常见的木马非常相似，主要有以下几种。

- 网页木马传播。攻击者在有漏洞的网站挂马，当用户不小心访问恶意网站时，勒索软件会被浏览器自动下载并在后台运行。
- 捆绑传播。与其他恶意软件捆绑发布进行传播。
- 电子邮件附件传播。攻击者通常会大肆收集邮箱地址，然后向这些邮箱发送带有病毒附件的邮件，作为电子邮件附件传播。
- 可移动存储介质传播。恶意软件借助可移动存储介质等自我复制到本地驱动器的根目录中，并成为具有隐藏属性和系统属性的可执行文件。

• 社交网络传播。勒索软件以社交网络中的图像等形式或其他恶意文件载体传播。

6. 其他恶意软件

(1) 逻辑炸弹(logic bomb)。逻辑炸弹是合法的应用程序，只是在编程时被故意写入了某种恶意功能，在一定情况下(如时间、次数或者某种逻辑组合)会出现。例如，作为版权保护方案，某个应用程序有可能会在运行几次后就在硬盘中将其自身删除。

(2) 点滴器(dropper)。点滴器也被称为投放器，是专为传送和安装其他恶意代码而设计的程序，它本身不具有直接的感染性和破坏性。

(3) 繁殖器(generator)。繁殖器是为制造恶意代码而设计的程序，一般会把某些已经设计好的恶意代码模块按照使用者的选择组合起来。

(4) 恶作剧(hoax)。恶作剧通常是为欺骗使用者而设计的程序，它会侮辱使用者或让其做出不明智的举动。

6.3.2 恶意代码基本原理

恶意代码的行为表现各异，破坏程度也不同，但基本作用机制大体相同，其整个作用过程可以分为以下 6 部分。

(1) 侵入系统。侵入系统是恶意代码实现其恶意目的的必要条件。恶意代码入侵的途径很多。例如：从互联网下载的程序本身就可能含有恶意代码；接收已经感染恶意代码的电子邮件；从光盘或 U 盘往系统上安装软件；黑客或者攻击者故意将恶意代码植入系统等。

(2) 维持或提升现有特权。恶意代码的传播与破坏必须盗用用户或者进程的合法权限才能完成。

(3) 实施隐蔽策略。为了不让系统发现恶意代码已经侵入系统，恶意代码可能会改名、删除源文件或修改系统的安全策略来隐藏自己。

(4) 潜伏。恶意代码侵入系统后，等待一定的条件，并具有足够的权限时，就会发作并进行破坏活动。

(5) 破坏。恶意代码的本质具有破坏性，其目的是造成信息丢失、泄密，破坏系统完整性等。

(6) 重复(1)～(5)对新的目标实施攻击过程。

恶意代码攻击模型如图 6.11 所示。

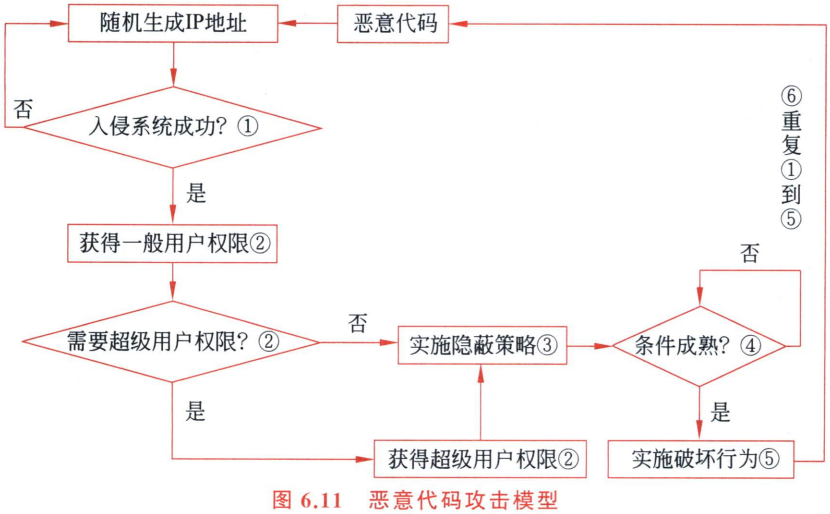


图 6.11 恶意代码攻击模型

6.3.2.1 恶意代码生存技术

恶意代码为了对抗反恶意代码软件的检测，通常采用反跟踪、加密、模糊变换(多态)、自动生产等技术，保证不被检测软件删除。

1. 反跟踪技术

恶意代码采用反跟踪技术可以提高自身的伪装能力和防破译能力，这增加检测与清除恶意代码的难度。目前常用的反跟踪技术有两类：反动态跟踪技术和反静态分析技术。

反动态跟踪技术主要包括以下几种。

(1) 禁止跟踪中断。针对调试分析工具运行系统的单步中断和断点中断服务程序，恶意代码通过修改中断服务程序的入口地址实现其反跟踪目的。1575 计算机病毒采用该方法将堆栈指针指向处于中断向量表中的 INT 0～INT 3 区域，阻止调试工具对其代码进行跟踪。

(2) 封锁键盘输入和屏幕显示，破坏各种跟踪调试工具运行的必需环境。

(3) 检测跟踪法。检测跟踪调试时和正常执行时的运行环境、中断入口和时间的差异，根据这些差异采取一定的措施，实现其反跟踪目的。例如，通过操作系统的 API 函数试图打开调试器的驱动程序句柄，检测调试器是否激活，确定代码是否继续运行。

(4) 其他反跟踪技术，如指令流队列法和逆指令流法等。

反静态分析技术主要包括以下几种。

(1) 对程序代码分块加密执行。为了防止程序代码通过反汇编进行静态分析，程序代码以分块的密文形式装入内存，在执行时由解密程序进行译码，某一段代码执行完毕后立即清除，保证任何时刻分析者不可能从内存中得到完整的执行代码。

(2) 伪指令法(junk code)。伪指令法指在指令流中插入“废指令”，使静态反汇编无法得到全部正常的指令，不能有效地进行静态分析。例如，Apparition 是一种基于编译器变形的 Win32 平台的病毒，编译器每次编译出新的病毒体可执行代码时都要插入大量的伪指令，既达到了变形的效果，也实现了反跟踪的目的。此外，伪指令技术还广泛应用于宏病毒与脚本恶意代码之中。

2. 加密技术

加密技术是恶意代码自我保护的一种手段，加密技术和反跟踪技术的配合使用，使得分析者无法正常调试和阅读恶意代码，不知道恶意代码的工作原理，也无法抽取特征串。

从加密的内容上划分，加密手段分为三种：信息加密、数据加密和程序代码加密。大多数恶意代码对程序本身加密，另有少数恶意代码对被感染的文件加密。例如，Cascade 是第一例采用加密技术的 DOS 环境下的恶意代码，它有稳定的解密器，可以解密内存中加密的程序体。Mad 和 Zombie 是 Cascade 加密技术的延伸，使恶意代码加密技术走向 32 位的操作系统平台。

3. 模糊变换技术(多态技术)

利用模糊变换技术，恶意代码每感染一个客体对象时，潜入宿主程序的代码互不相同。目前，模糊变换技术主要分为以下 5 种。

(1) 指令替换技术。模糊变换引擎(mutation engine)对恶意代码的二进制代码进行反汇编，解码每一条指令，计算出指令长度，并对指令进行同义变换。例如，将指令“XOR REG，REG”变换为“SUB REG，REG”；寄存器 REG1 和寄存器 REG2 进行互换；JMP 指令和 CALL 指令进行变换等。

(2) 指令压缩技术。模糊变换器检测恶意代码反汇编后的全部指令，对可进行压缩的一段指令进行同义压缩。但是压缩技术会改变病毒体代码的长度，因此需要对病毒体内的跳转指令

进行重定位。例如，指令“MOV REG,12345678 / ADD REG,87654321”变换为指令“MOV REG,99999999”；指令“MOV REG,12345678 / PUSHREG”变换为指令“PUSH 12345678”等。

（3）指令扩展技术。与指令压缩技术相反，指令扩展技术把每一条汇编指令进行同义扩展，所有压缩技术变换的指令都可以采用扩展技术实施逆变换。扩展技术变换的空间远比压缩技术大得多，有的指令可以有几十种甚至上百种的扩展变换。扩展技术同样要改变恶意代码的长度，也需要对恶意代码中跳转指令进行重定位。

（4）伪指令技术。伪指令技术主要是对恶意代码程序体中插入无效指令，如空指令、JMP下一指令和指令“PUSH REG/MOV REG,12345678/POP REG”等。

（5）重编译技术。采用重编译技术的恶意代码中携带恶意代码的源码，需要自带编译器或操作系统提供编译器进行重新编译，这种技术既实现了变形的目的，也为跨平台的恶意代码出现打下了基础。尤其是各类 UNIX/Linux 操作系统，系统默认配置有标准 C 语言的编译器。宏病毒和脚本恶意代码是典型的采用这类技术变形的恶意代码。造成全球范围传播和破坏的第一例变形病毒是 Tequtla，从该病毒的出现到编制出能够检测该病毒的软件，研究人员花费了 9 个月的时间。

4. 自动生产技术

恶意代码自动生产技术主要针对人工分析检测技术，利用多态变换引擎简单实现恶意代码的组合和变化，使程序代码本身发生变化，并保持原有功能。例如，即使对计算机病毒一无所知的用户，利用“计算机病毒生成器”也能组合出算法不同、功能各异的计算机病毒；另外，“多态性发生器”可将普通病毒编译成复杂多变的多态性病毒。保加利亚的 Dark Avenger 是较为著名的一个例子，这个变换引擎每产生一个恶意代码，其程序体都会发生变化，反恶意代码软件如果采用基于特征的扫描技术，根本无法检测和清除这种恶意代码。

6.3.2.2 恶意代码攻击技术

常见的恶意代码攻击技术包括：进程注入、三线程、端口复用、超级管理、端口反向连接以及缓冲区溢出攻击。

1. 进程注入技术

当前的操作系统中都有系统服务和网络服务，它们都在系统启动时自动加载。进程注入技术就是将这些与服务相关的可执行代码作为载体，恶意代码程序将自身嵌入到这些可执行代码之中，实现自身隐藏和启动的目的。

这种形式的恶意代码只需安装一次，以后就会被自动加载到可执行文件的进程中，并且会被多个服务加载。只有系统关闭时，服务才会结束，所以恶意代码程序在系统运行时始终保持激活状态。比如，恶意代码 WinEggDropShell 可以注入 Windows 下的大部分服务程序。

2. 三线程技术

在 Windows 操作系统中引入了线程的概念，一个进程可以同时拥有多个并发线程。三线程技术就是一个恶意代码进程同时开启了三个线程，其中一个为主线程，负责远程控制的工作；另外两个辅助线程是监视线程和守护线程，监视线程负责检查恶意代码程序是否被删除或被停止自启动。

守护线程注入其他可执行文件内，与恶意代码进程同步，一旦进程被停止，它就会重新启动该进程，并向主线程提供必要的数据，这样就能保证恶意代码运行的可持续性。例如，“中国黑客”等就是采用这种技术的恶意代码。

3. 端口复用技术

端口复用技术指重复利用系统网络打开的端口(如25、80、135和139等常用端口)传送数据,这样既可以欺骗防火墙,又可以少开新端口。端口复用是在保证端口默认服务正常工作的条件下复用,具有很强的欺骗性。例如,木马Executor就是利用80端口传递控制信息和数据,实现其远程控制的目的。

4. 超级管理技术

一些恶意代码还具有攻击反恶意代码软件的能力。为了对抗反恶意代码软件,恶意代码采用超级管理技术对反恶意代码软件系统进行拒绝服务攻击,使反恶意代码软件无法正常运行。例如,"广外女生"是一个国产的木马,它采用超级管理技术对"金山毒霸"和"天网防火墙"进行拒绝服务攻击。

5. 端口反向连接技术

防火墙对于外部网络进入内部网络的数据流有严格的访问控制策略,但对于从内网到外网的数据却疏于防范。端口反向连接技术指让恶意代码攻击的服务端(被控制端)主动连接客户端(控制端)。国外的Boinet是最先实现这项技术的木马程序,它可以通过ICO、IRC、HTTP和反向主动连接这4种方式联系客户端。国内最早实现端口反向连接技术的恶意代码是"网络神偷"。"灰鸽子"则是这项技术的集大成者,它内置了FTP、域名、服务端主动连接这3种服务端在线通知功能。

6. 缓冲区溢出攻击

缓冲区溢出攻击占远程网络攻击的80%。缓冲区溢出攻击也成为恶意代码从被动式传播转为主动式传播的主要途径。例如,"红色代码"利用IIS Server上Indexing Service的缓冲区溢出漏洞完成攻击、传播和破坏等恶意目的。"尼姆达蠕虫"利用IIS 4.0/5.0 DirectoryTraversal的弱点,以及红色代码Ⅱ所留下的后门,完成其传播过程。

6.3.2.3 恶意代码隐藏技术

恶意代码隐藏技术通常包括本地隐藏和通信隐藏。其中本地隐藏主要有文件隐藏、进程隐藏、网络连接隐藏、编译器隐藏、Rootkit隐藏等;通信隐藏主要包括通信内容隐藏和传输通道隐藏。

1. 本地隐藏

本地隐藏是为了防止本地系统管理人员觉察而采取的隐藏手段,有如下分类。

(1) 文件隐藏:通过定制文件名,使恶意代码的文件更名为系统的合法程序文件名,或者将恶意代码文件附加到合法程序文件中。

(2) 进程隐藏:恶意代码通过附着或替换系统进程,使恶意代码以合法服务的身份运行,这样可以很好地隐蔽恶意代码。

(3) 网络连接隐藏:恶意代码可以复用现有服务的端口来实现网络连接隐蔽,将自己的数据包设置特殊标识,通过标识识别连接信息。

(4) 编译器隐藏:使用该方法可以实施原始分发攻击,恶意代码的植入者是编译器开发人员。其包括针对特定程序的恶意代码和针对编译器的恶意代码。

(5) Rootkit隐藏:Windows操作系统中的Rootkit分为两类,用户模式下的Rootkit和内核模式下的Rootkit。用户模式下的Rootkit修改二进制文件,或者修改内存中的一些进程,同时保留它们受到限制地通过API访问系统资源的能力,需要的特权小,更轻便,用途也多种多样,它隐藏自己的方式是修改可能发现自己的进程。内核模式下的Rootkit隐蔽性更

好，且它能直接修改更底层的系统功能。

2. 通信隐藏

通信内容隐藏主要依靠加密技术保护通信内容，但无法隐藏通信状态。传输通道隐藏主要采用隐蔽通道技术，隐蔽通道是允许进程违反系统安全策略传输信息的通道。

6.3.3 恶意代码分析技术

在检测和清除恶意代码的最初阶段，恶意代码以计算机病毒为主，数量少且传播速度缓慢，编写反病毒等安全防护软件并不困难。随着恶意代码的层出不穷，恶意代码的自我保护技术越来越复杂，行为具有不确定性。尽管恶意代码以许多不同的形态出现，但分析恶意代码的技术是通用的。

恶意代码分析指利用多种分析工具掌握恶意代码样本程序的行为特征，了解其运行方式及安全危害，是准确检测和清除恶意代码的关键环节。恶意代码分析技术是网络应急响应的基础。通过对主机状态信息的检测和分析，了解恶意代码的基本功能，掌握恶意代码对受害系统可能进行的破坏活动，为受害主机的系统恢复、系统损失评估提供信息，掌握恶意代码的攻击技术、自启动技术、删除保护技术等，为防御和清除恶意代码提供信息。

目前，常用恶意代码的分析方法可以分为静态分析和动态分析两种。静态分析方法是在没有运行恶意代码时对其进行分析的技术，而动态分析方法则需要运行恶意代码。这两种方法结合使用，能较为全面地收集恶意代码的相关信息，以达到更好的分析效果。

1. 静态分析技术

恶意代码静态分析技术，是在不执行恶意代码的前提下进行分析，不会对分析系统造成破坏，方法包括反汇编分析、源代码分析、二进制统计分析等，属于逆向工程分析方法。

这种方法早期表现为静态特征码扫描技术，广泛应用于反病毒领域，技术也比较成熟，这种方法的最大工作量在于特征码的提取和分析。通过静态分析法可以分析出恶意代码的大致结构，使用的系统调用，采用的技巧，如何将恶意代码的破坏行为转换成恶意代码清除行为，哪些代码可被用作恶意代码的特征码，以及如何预防这种恶意代码。

常见的静态分析方法有以下四类。

(1) 反恶意代码软件分析。利用反病毒工具来检测各种形式的恶意代码，反病毒软件通过特征代码法、校验和法、软件模拟法等方法检测出恶意代码。如果反病毒软件已经收录了该恶意代码的分析数据，那么可以直接利用它们的分析结果。如果没有收录该恶意代码的分析数据，就根据恶意代码的信息包括名字及其他的一些特点，通过互联网搜寻更多的资料。

(2) 字符串分析。字符串分析的目的是寻找恶意代码文件中使用的 ASCII 或其他方法编码的连续字符串。很多恶意代码程序中包含了一些字符串，这些字符串涉及代码使用的各种库和程序。通常可以在恶意代码样本中搜寻以下信息：①恶意代码的名字；②帮助和命令行选项；③用户对话框，可以通过它分析恶意代码的目的；④后门密码；⑤恶意代码相关的网址；⑥恶意代码作者或者攻击者的 E-mail 地址；⑦恶意代码用到的库，函数调用及其他可执行文件；⑧其他的有用信息。

(3) 脚本分析。如果恶意代码采用 JavaScript、VBScripts 或是 Shell 等脚本语言编写，就可通过文本编辑器将脚本打开查看源代码。脚本分析能帮助分析者短时间内识别大多数流行的脚本类型。通过分析源代码来理解程序的功能、流程、逻辑判定及程序的企图等。

(4) 反汇编分析。反汇编分析指分析人员借助反汇编工具来对恶意代码样本进行反汇

编，从反汇编出来的程序清单上根据汇编指令码和提示信息着手分析。

通常，恶意代码样本的存在形式是可执行文件、动态链接库、软件库或其他形式的文件，而这些文件在标准的文本编辑器中则以乱码显示，无法查看。编译器在编译时，把源代码转换为可执行代码时，产生二进制数据如指令操作码、文本、标识符等，并以目标文件的形式保存。经过编译和链接过程产生的二进制文件中存留了大量的有用信息，反汇编工具可以借助这些有用信息将二进制可执行代码转换为汇编语言指令。

静态反汇编中常用的调试工具有 W32DSAM、IDA Pro 等，这些反汇编工具具备可快速到达指定的代码位置、可看到 JMP 命令跳转的位置、可看到参考字符串、可保存静态汇编代码等功能。

2. 动态分析技术

动态分析法是通过监视恶意代码运行过程来分析恶意代码的方法。动态分析法可根据是否分析代码语义分为动态跟踪法和外部监测法。

(1) 动态跟踪法：利用程序调试工具单步执行恶意代码，进行动态跟踪分析。常用的工具有 OllyDbg、SoftIce 等。一般分为两步：第一步，先对恶意代码进行粗跟踪，即大块跟踪，遇到调用指令 CALL、重复操作指令 REP、循环操作指令 LOOP 等，一般不需跟踪进去，而是根据执行结果分析该段程序的功能；第二步，对关键部分进行细跟踪，即针对性地对关键代码段进行具体而详细的跟踪分析。一般情况下，可能要反复进行若干次才能读懂该程序，每次要把比较关键的中间结果或指令地址记录下来，从而对下一次分析起到更大的帮助作用。

(2) 外部监测法：在恶意代码执行过程中，对恶意代码的行为进行监测，分析恶意代码运行过程系统发生的变化。包括进程、文件、注册表、启动项、网络通信等方面。

恶意代码行为动态分析的核心在于挂钩(hook)技术，利用挂钩技术监视恶意代码样本在整个执行过程中的系统调用和 API 函数使用状态来检测和分析恶意代码样本的功能。

3. 恶意代码分析工具

无论静态分析还是动态分析方法，都可以利用一些工具辅助分析，具体可以分为以下三类。

(1) 虚拟执行环境：进行恶意代码分析具有很大的风险性，如果操作不当，会对分析系统造成严重破坏，所以需要在虚拟的相对安全的环境中分析恶意程序。虚拟执行环境包括 VMWare、沙盘(sandboxie)等。

(2) 静态分析工具：静态分析工具主要用于对可移植可执行(PE)文件或脚本文件的特征进行分析。包括文本及十六进制阅读及编辑工具(记事本、UltraEdit)，PE 文件格式处理工具(Stud_PE、Peid 等)，反汇编工具(IDA 等)。

(3) 动态分析工具：动态分析工具主要用于对恶意代码动态调试跟踪和外部监测。动态调试工具包括 OllyDbg、SoftIce，外部监测工具有 ProcessMon、RegMon、FileMon、TcpView、Ethereal 等。

6.4 软件安全测试

软件测试是生产高质量软件必不可少的一个工程实践活动，它伴随着软件的产生而出现，是保证软件质量的关键手段。早期的软件开发过程中软件规模都很小、复杂程度低，软件开发的过程混乱无序，因此测试的含义比较狭窄，开发人员通常将测试等同于“调试”，目的是纠正

软件中已经知道的故障，且常常由开发人员自己完成测试工作。

到了20世纪80年代初期，软件和IT行业进入了大发展时期，软件趋向大型化、高复杂度，软件的质量越来越重要。一些软件测试的基础理论和实用技术开始形成，软件测试有了行业标准(IEEE/ANSI)，1983年IEEE提出的软件工程术语中给出软件测试定义："使用人工或自动的手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的需求或弄清预期结果与实际结果之间的差别"。这个定义明确指出：软件测试的目的是检验软件系统是否满足需求。它再也不是一个一次性的、开发后期的活动，而是与整个开发流程融合成一体。软件测试的根本目标是尽可能多地发现并排除软件中潜藏的错误，最终把一个高质量的软件系统交给用户使用。

随着软件技术的发展，安全性作为软件的质量属性受到越来越多的重视，针对安全问题的测试越来越重要。软件安全性测试是确定软件的安全特性实现是否与预期设计一致的过程，是验证软件安全等级和识别潜在安全缺陷的过程，检查软件系统是否拥有一定程度的保护机制和防止受到非法侵入的能力。

安全性测试与传统软件测试最大的区别在于：前者强调软件不应当做什么，后者则更多强调确保软件能够完成预先设计的功能。传统软件测试重点在于功能性测试，验证用户的输入是否能够得到正确的输出，如果输出不正确则说明软件存在缺陷，但不一定是安全性缺陷，即软件漏洞。安全性测试则更注重发现软件漏洞，验证用户输入是否会导致不安全的事件发生，如导致缓冲区溢出、未授权用户越权访问系统数据等。

本节主要针对软件安全测试进行概述，在分类阐述软件测试方法的基础上，介绍几种主要的软件安全测试方法。

6.4.1 软件测试方法分类

传统的用于发现软件缺陷的功能性测试方法很多，黑盒测试、白盒测试、静态测试、动态测试等都是用于描述测试方式的常见术语。软件安全测试可以进一步分为安全功能测试和安全漏洞测试。软件功能性测试和安全性测试均可以分阶段进行，每个阶段的测试目的和方法手段也各不相同。

1. 软件测试阶段

软件测试的阶段通常分为单元测试、集成测试、确认测试和系统测试，如图6.12所示，在实际项目开发过程中，软件测试各阶段工作可以与软件开发生命周期各阶段有效结合。

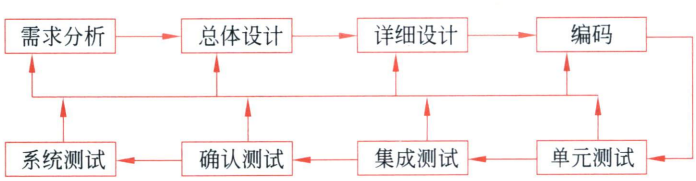


图6.12 软件测试阶段

(1) 单元测试：对用软件的每一个程序单元进行测试，检查各个程序模块的正确性，并配合适当的代码审查。单元测试通常是最小规模的测试，由于需要知道内部程序设计和编码的细节知识，经常由程序员而非测试员来进行，待测试软件系统的体系结构良好，有助于做好这一阶段的测试工作。

(2) 集成测试：把已测试过的模块组装起来，以便发现与接口有关的问题，如数据模块间

传递、模块组合性能、模块调用性能等。模块可以是代码块，也可以是独立的应用，以及客户端或服务器端程序。集成测试的对象是已经经过单元测试的各模块，主要目的是确定一个软件系统的各模块能否在一起共同工作。

(3) 确认测试：检查软件是否满足了需求规格说明书中的各种需求，以及软件配置是否完全、正确。确认测试又称为验收测试，目的是验证软件的有效性。

(4) 系统测试：把已经通过验收的软件，放入实际运行环境中运行；用户记录在测试过程中遇到的一切问题，定期报告给开发者。

2. 传统软件测试方法

传统软件测试方法根据软件内部结构的可见性可以分为黑盒测试、白盒测试，从测试过程中是否需要软件执行其测试方法可以分为静态测试和动态测试。

(1) 黑盒测试：又称为功能测试。用该方法进行测试时，被测程序被当作一个“黑盒”，测试者无须知道程序内部结构，只需要知道程序的输入及输出是否和预期输出相符。黑盒测试的主要依据是“需求”，在独立测试阶段多采用黑盒测试方法。

(2) 白盒测试：又称结构测试或逻辑驱动测试。用该方法进行测试时，测试者必须了解被测程序的内部结构，根据被测程序的内部构造设计测试用例。在白盒测试的过程中，需要测试用例的设计对被测程序的结构做到一定程度的覆盖。白盒测试的主要依据是“设计”，更适合在单元测试中运用。

(3) 静态测试：不实际运行被测试的软件，而是对软件进行分析、检查和审阅，来寻找逻辑错误。静态测试是软件开发中十分有效的质量控制方法之一，特别是在软件开发生命周期的早期和中期阶段实施静态测试非常有效。开发初期的工作质量可能直接关系到软件开发成本，由于程序还没有编出来，可以直接运行的代码尚未产生，此时可以采用静态测试方法对设计的一些思路进行检查或审核，保证开发质量。

(4) 动态测试：动态测试和静态测试相反，在测试的过程中，实际运行软件，检测软件的动态行为和运行结果的正确性。动态测试包括两个基本要素：一是被测软件；二是在软件运行过程中的输入数据，每一次测试需要的测试数据称为测试用例。因此，动态测试一般在软件编码阶段完成之后进行。由于其比较强的错误检测能力，动态测试得到了广泛的采用。

3. 软件安全性测试

传统软件测试并不用来发现软件中的安全缺陷，不能用来验证软件安全性。因此，需要考虑采用专门的安全性测试来保证软件安全性。事实上，软件安全性测试是在充分考虑软件安全性问题的前提下进行的测试，有着与传统测试不同的关注重点。除上述传统软件测试方法和手段外，需要将安全性测试整合到软件开发生命周期各阶段的测试工作中，在软件的生命周期内采取一系列措施，防止出现有违反安全策略的异常情况和在软件的设计、开发、部署、升级及维护过程中的潜在系统漏洞。

如图 6.13 所示，在项目开发过程中，在项目设计阶段实施安全分析，检验软件需求规格说明中规定的防止危险状态措施的有效性，以及在每一个危险状态下的反应。此外，在单元测试、集成测试、验收测试等工作阶段，进行针对性的安全测试。例如，对安全性关键的软件单元和软件部件，单独进行加强的安全性测试，用以确认其满足安全性需求；或是用错误的安全性关键操作进行测试，以验证系统对这些操作错误的反应；还可以在异常条件下测试软件，以表明不会因可能的单个或多个输入错误而导致不安全状态。

安全功能测试主要基于软件的安全功能需求说明，测试软件的安全功能实现是否与安全

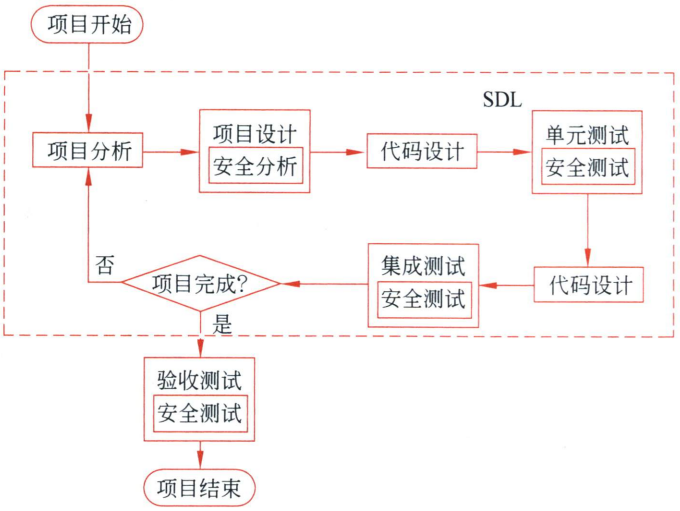


图 6.13 软件安全性测试阶段

需求一致，需求实现是否完备。其中，安全功能需求主要包括数据机密性、完整性、可用性、不可否认性、身份认证、授权、访问控制、审计跟踪、委托、隐私保护、安全管理等。

安全漏洞测试则从攻击者的角度，以发现软件的安全漏洞为目的，找出系统在设计、实现、操作、管理上存在的可被利用的缺陷或弱点。漏洞被利用可能造成软件受到攻击，使软件进入不安全的状态，安全漏洞测试就是识别软件的安全漏洞。

尽管传统软件测试方法重点在软件功能测试，但是通过组织合适的测试步骤和适当的测试用例，也能对软件的安全质量进行测试，可以采用的测试方法包括常见的白盒测试、黑盒测试和灰盒测试等。在传统测试之外，模糊测试和渗透测试是常用的软件安全性测试方法。

如表 6.4 所示，软件安全性测试可以采用黑盒测试结合静态分析方法进行功能验证，通过检查各类需求文档、需求规格说明书等，对于涉及安全的软件功能（如用户管理模块、权限管理、加密系统、认证系统等）进行测试，验证这些安全功能是否有效；还可以通过动态模拟攻击来验证软件系统的安全防护能力，实施模糊测试或渗透测试，来发现软件中可能存在的安全漏洞。同样，白盒测试方法与静态和动态测试相结合，对源代码、二进制代码等既可以进行静态分析或审核，也可以在软件运行时进行动态监测。

表 6.4 软件安全性测试手段

	静态测试	动态测试
黑盒测试	检查产品或市场需求文档及需求规格说明书	模糊测试，渗透测试，模拟错误注入攻击进行验证
白盒测试	审查代码	调试，单元测试

6.4.2 白盒、黑盒、灰盒测试

白盒测试方法具体是根据模块内部结构，基于内部逻辑结构，针对程序语句、路径、变量状态等来进行测试，检验程序中的各个分支条件是否得到满足，每条执行路径是否按预定要求正确地工作。如图 6.14 所示，对程序内部结构、代码分支和条件一目了然，可以通过设计测试用例，使测试经过每一个条件和分支。然而，对一个具有多重选择和循环嵌套的程序，不同的路

径数目可能是天文数字。假设图 6.14 中的示例程序包括一个执行 20 次的循环，则包含的不同执行路径数达 5^{20} 条，对每一条路径进行测试需要 1 ms，一年工作 365×24 小时，把所有路径测试完需要 3170 年。可以看出，对于一个复杂的软件系统（可能是几十万行甚至几百万行代码），全面地完成逻辑分析不现实，所以白盒测试方法非常适合进行单元测试。

此外，传统白盒测试方法虽然测试覆盖了第一行语句、所有条件和分支等，可以保证程序在逻辑、处理和计算上没有问题，但不能保证在产品功能特性上没有问题。所以，单元测试需要借助黑盒测试方法来对单元实现的功能特性进行检验。

黑盒测试方法把程序看作一个黑盒子，不考虑程序内部结构和内部特性，而是考查数据的输入、条件限制和数据输出。如图 6.15 所示，黑盒测试方法通过不同的数据输入，获得输出结果，从而检验程序的实际行为是否与产品规格说明、客户的需求保持一致。具体地，黑盒测试根据用户的需求和已经定义好的产品规格，针对程序接口和用户界面进行测试，检验程序是否能适当地接收输入数据而产生正确的输出信息，并且保持外部信息（如数据库或文件）的完整性。用黑盒测试发现程序中的错误，必须在所有可能的输入条件和输出条件中确定测试数据，来检查程序是否都能产生正确的输出，但这是不可能的。举例来说，假设程序 P 有输入整数 X 和 Y 及输出量 Z，在字长为 32 位的计算机上运行，对其进行黑盒测试时采用的测试数据组可能达到 2^{64}，如果测试一组数据需要 1 ms，一年工作 365×24 小时，完成所有测试需要 5 亿年。

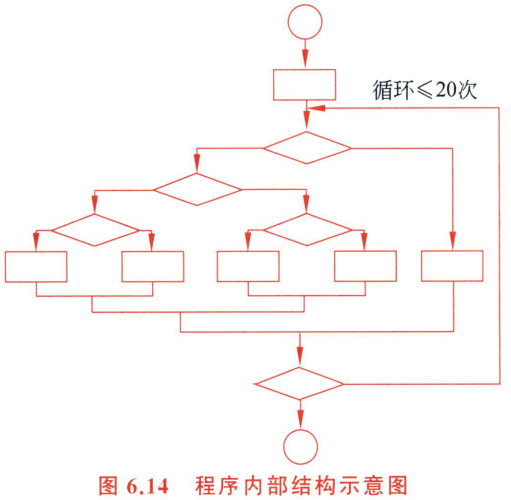


图 6.14 程序内部结构示意图

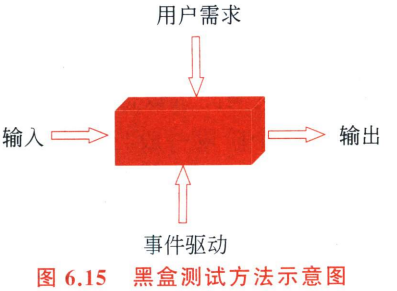


图 6.15 黑盒测试方法示意图

由此可看出，黑盒白盒两种测试方法并不能保证测试工作高效地进行。并且，软件测试的根本目的是要提高应用软件的质量，保证其未来能够可靠稳定运行。白盒测试的代码扫描和应用功能挂不上钩，而黑盒测试的结果通常是测试人员把功能测试发现的问题提交给开发人员处理。这种问题处理方式也非常没有效率，会延缓问题的处理时间。因此，有了介于两者之间的灰盒测试。

灰盒测试，是介于白盒测试与黑盒测试之间的一种测试方法，多用于集成测试阶段，不仅关注输入、输出的正确性，同时也关注程序内部的情况。比较而言，灰盒测试不像白盒测试那样详细、完整，但又比黑盒测试更关注程序的内部逻辑，常常是通过一些表征性的现象、事件、标志来判断内部的运行状态，一旦发现问题，可以迅速定位到黑盒中的某个分支，甚至代码级别。灰盒测试是另辟新径的一种测试方法，可以增强测试效率及错误发现和错误分析的效率，但要真正发挥其作用，还需要相应的自动化工具。在目前有许多软件测试工具可利用的条件

下，灰盒测试法的自动化程度可达70%～90%。

6.4.3 模糊测试

模糊测试(fuzz testing)是一种基于缺陷注入的自动化软件测试技术，它使用大量半有效的数据作为应用程序的输入，以程序是否出现异常或错误作为标志，来发现应用程序中可能存在的安全漏洞。半有效的数据指对应用程序来说，测试用例的必要标识部分和大部分数据是有效的，但同时该数据的其他部分是无效或存在漏洞攻击的。因为半有效数据会被待测程序认为这是一个有效的数据，这样应用程序就有可能发生错误，从而导致应用程序的崩溃或触发相应的安全漏洞。可以看出，模糊测试实际上就是一种通过向目标系统提供非预期的输入并监视异常结果来发现软件漏洞的方法。

模糊测试执行过程主要包含以下几个基本阶段。

(1) 识别测试目标：确定明确的测试目标，才能决定使用的模糊测试工具或方法。比如，需要选择应用包含的特定文件或者库作为测试目标，需要把注意力放在多个应用程序之间共享的那些二进制代码上。因为如果这些共享的二进制代码中存在安全漏洞，将会有非常多的用户受到影响，因而风险也更大。

(2) 识别输入：大部分可被利用的安全漏洞都是由于应用没有对用户的输入进行校验或是进行必要的非法输入处理。是否能找到所有的输入向量(input vector)是模糊测试能否成功的关键。寻找输入向量的原则是：从客户端向目标应用发送的任何内容，包括头(headers)、文件名(file name)、环境变量(environment variables)、注册表键(registry keys)，以及其他信息，都可能是潜在的模糊测试变量。

(3) 生成模糊测试数据：大多数模糊测试的方法是通过向目标系统不断输入可以诱发软件缺陷的测试数据，因此测试数据的生成是模糊测试非常关键的环节，主要依赖测试目标系统的特点和数据格式进行生成。

(4) 执行模糊测试数据：自动化地向被测的系统发送数据包、打开文件或是执行被测应用的过程。这个阶段一般与生成测试数据并行进行。

(5) 监视异常：监控异常和错误是模糊测试中重要但经常容易被忽略的步骤。模糊测试需要根据被测应用和所决定采用的模糊测试类型来设置各种形式的监视。

(6) 判定发现的漏洞是否可被利用：当在模糊测试中发现了一个错误时，需要判定这个被发现的错误是否是一个可被利用的安全漏洞。这种判定过程既可以由模糊测试的执行者来进行，也可以交给安全测试专家来进行。

模糊测试技术的应用十分广泛，可以测试的对象种类繁多，如环境变量和参数、Web应用程序、文件格式、网络协议、Web浏览器等方面，如表6.5所示。

表6.5 模糊测试对象分类

测试对象	特　点	常用测试工具
环境变量和参数	最简单的模糊测试，即对命令行参数或环境变量进行恶意注入，注入成功后有可能导致程序崩溃	iFuzz
Web应用程序	特别关注遵循HTTP协议规范的测试数据包，针对拒绝服务、跨站点编写脚本、SQL注入和弱访问控制等漏洞攻击进行测试	Wfuzz(开源)
文件格式	发现应用程序(如Web浏览器、邮件服务器、Office办公组件及媒体播放器等)在解析特定文件格式时出现的漏洞	FileFuzz(Windows平台)，SPIKEfile(UNIX平台)

续表

测试对象	特　点	常用测试工具
网络协议	最广泛被利用的模糊测试类别，发现邮件、数据库、远程访问、多媒体和备份等服务器很多高风险漏洞	PROTOS
Web浏览器	通过刷新HTML页、加载Web页和目标单浏览器对象等手段，发现DoS、缓冲区溢出、远程命令执行、绕过跨越限制、绕过安全区和地址栏欺骗等漏洞	COMRaider

6.4.4 渗透测试

渗透测试(penetration testing)是通过模拟恶意黑客的攻击方法，来评估计算机网络系统安全的一种测试方法。测试过程包括对系统的任何弱点、技术缺陷或漏洞的主动分析，渗透测试人员在不同的位置(如内网、外网等位置)利用黑客可能使用的各种攻击技术和漏洞发现技术，发现和挖掘系统中存在的漏洞，然后输出渗透测试报告，并提交给网络所有者。网络所有者根据渗透人员提供的渗透测试报告，可以清晰知晓系统中存在的安全隐患和问题。因此，渗透测试能直观地展现整个系统(不局限于软件)所面临的安全问题。

渗透测试利用网络安全扫描器、专用安全测试工具和富有经验的渗透测试工程师的人工经验，对网络中的核心服务器及重要的网络设备(包括服务器、网络设备、防火墙等)进行非破坏性质的模拟黑客攻击，目的是侵入系统并将入侵的过程和细节产生报告给用户。

通常一次完整的渗透测试可以分为收集、扫描、漏洞利用和后维持攻击4个阶段。首先收集信息，抽取出对网络渗透有用的信息，制定渗透策略并进行漏洞测试，模拟攻击的真实过程，最后整理收集到的信息并提交测试报告。安全业界普遍认同的渗透测试执行标准(Penetration Testing Execution Standard，PTES)对渗透测试过程进行了标准化，将其分为以下7个阶段。

1. 前期交互阶段

在前期交互(pre-engagement interaction)阶段，渗透测试团队与客户组织交互讨论，重点是确定渗透测试的范围、目标、限制条件及服务合同细节。此阶段通常涉及收集客户需求、准备测试计划、定义测试范围与边界、定义业务目标、项目管理与规划等活动。

2. 情报搜集阶段

目标范围确定后，进入情报搜集(information gathering)阶段，渗透测试团队可以利用各种信息来源与搜集技术方法，尝试获取更多关于目标组织网络拓扑、系统配置与安全防御措施的信息。

渗透测试人员可以使用的情报搜集方法包括公开来源信息查询、Google Hacking、社会工程学、网络踩点、扫描探测、被动监听、服务检查点等。情报搜集是渗透测试中最重要的一环，而对目标系统的情报探查能力是渗透测试者一项非常重要的技能，情报搜集是否充分在很大程度上决定了渗透测试的成败，如果遗漏关键的情报信息，会导致在后面的阶段一无所获。

3. 威胁建模阶段

搜集到充分的情报信息之后，渗透测试团队针对获取的信息进行威胁建模(threat modeling)，进行攻击规划。这是渗透测试过程中非常重要，但很容易被忽视的一个关键点。

通常情况下，即使是小规模的侦查工作也能收集海量据，情报收集过程结束后，对目标就有了十分清楚的认识，包括公司组织的架构、内部部署的技术等，据此进行威胁建模，确定攻击

方案，有利于后面各阶段测试工作的开展。

4. 漏洞分析阶段

威胁建模完成后，接下来考虑如何取得目标系统的访问控制权，即漏洞分析（vulnerability analysis）阶段。

在这一阶段，渗透测试人员需要综合分析前几个阶段获取并汇总的情报信息，特别是安全漏洞扫描结果、服务查点信息等，通过搜索可获取的渗透代码资源，找出可以实施渗透攻击的攻击点，并在实验环境中进行验证。此外，高水平的渗透测试团队还会针对攻击通道上的关键系统与服务进行安全漏洞探测与挖掘，期望找出可被利用的未知安全漏洞，并开发出渗透代码，从而发现攻击通道上的关键路径。

5. 渗透攻击阶段

渗透攻击（exploitation）是渗透测试过程中最具有“魅力”的环节。在此阶段，渗透测试团队需要利用他们所找出的目标系统安全漏洞，来真正入侵系统当中，获得访问控制权。

渗透攻击可以利用从公开渠道获取的渗透代码，但一般在实际应用场景中，渗透测试人员还需要充分地考虑目标系统特性来定制渗透攻击，并需要挫败目标网络与系统中实施的安全防御措施，才能成功达到渗透目的。

6. 后渗透攻击阶段

后渗透攻击（post exploitation）是整个渗透测试过程中最能够体现渗透测试团队创造力与技术能力的环节。前面几个阶段大多是按部就班地完成非常普遍的目标，而在后渗透测试阶段，需要渗透测试团队根据目标组织的业务经营模式、保护资产形式与安全防御计划的不同特点，自主设计出攻击目标，识别关键基础设施，并寻找客户组织最具价值和尝试安全保护的信息和资产，最终获得能够对客户组织造成最重要业务影响的攻击途径。

其与渗透攻击阶段的区别在于，后渗透攻击更加重视在渗透目标之后的进一步攻击行为。这一阶段主要支持在渗透攻击取得目标系统远程控制权之后，在受控系统中进行各式各样的后渗透攻击动作，如获取敏感信息、实施跳板攻击等。

7. 报告阶段

渗透测试团队最终要向客户组织提交描述渗透测试过程的渗透测试报告（reporting）。报告凝聚了之前所在阶段中渗透测试团队所获取的关键情报信息、探测和发掘出的系统安全漏洞、成功渗透攻击的过程，以及造成业务影响后果的攻击途径，同时还要站在防御者的角度上，帮助分析安全防御体系统中的薄弱环节、存在的问题，以及修补与升级技术方案。

6.4.5 软件安全测试工具

安全测试一直充满着挑战，安全测试工具一直没有绝对的标准。虽然有时会让专业的安全厂商来实施渗透测试、扫描，但考虑到控制、管理和商业秘密的原因，许多公司通常自己组织实施安全性测试，这就需要购买相关的安全性测试工具。本节重点介绍静态分析工具IDA PRO。

IDA PRO（IDA）是一个顶级的交互式反汇编工具，是安全渗透人员进行逆向安全测试的必备工具，其强大的静态反汇编和逆向调试功能能够帮助安全测试人员发现代码级别的高危致命安全漏洞。

可以被IDA解析的文件包括.exe、.so、.o等格式，在IDA中直接打开上述格式的文件即可，打开过程选择如图6.16所示。

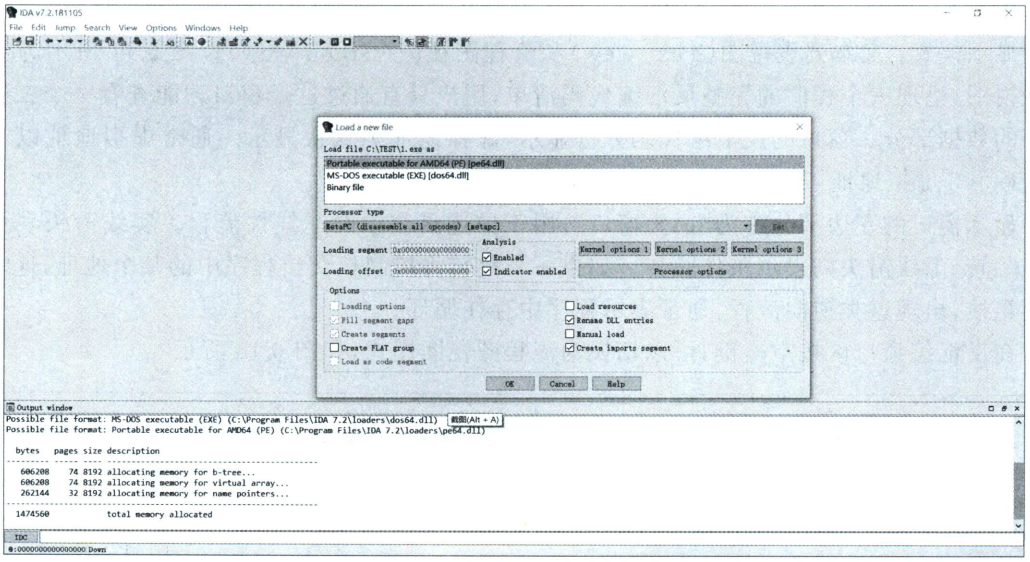


图 6.16　IDA 打开过程

IDA 提供了 3 种不同的打开方式：新建(New)，运行(Go)，上一个(Previous)。初次打开可以选择 Go 方式。进入之后，使用 File 中的 Open 选项打开文件。打开 IDA，主界面如图 6.17 所示。

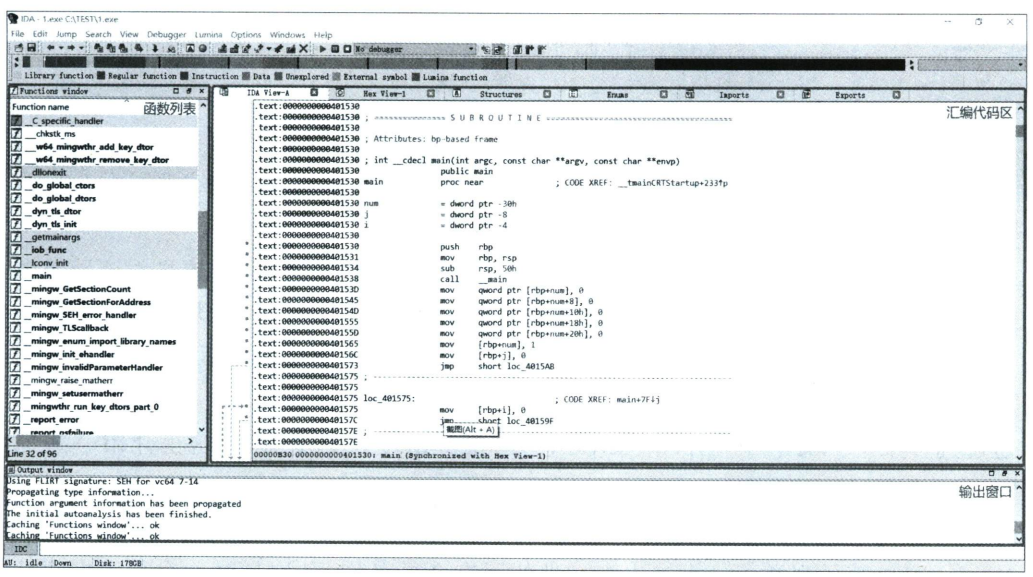


图 6.17　IDA 主界面

默认配置下，IDA 打开后，会出现 3 个立即可见的窗口：左侧窗口为函数列表，一个文件被反编译后所有的函数列表都可以在此窗格中显示；右侧占据主要区域的窗口为汇编代码区，双击每个函数，可以看到对应函数对应的汇编代码段；下方为输出窗口，文件反汇编过程中的信息都可以在此窗口中看到。

汇编代码区是最主要的工作区，是进行逆向分析的场所，工作区可以在多个窗口间进行切换，IDA View 是反汇编窗口，HexView 是十六进制格式显示的窗口，Structures 列举结构，Enums 列举枚举，Imports 是导入表(程序中调用的外部函数)，Exports 是导出表。

IDA View窗口也叫反汇编窗口，是操作和分析二进制文件的主要工具。反汇编窗口有两种显示模式：文本列表视图(Text view)和流程图视图(Graph view)。图6.17所示为文本列表视图，呈现一个程序的完整反汇编代码清单，用户只有通过这个窗口才能查看一个二进制文件的数据部分。窗口的反汇编代码分行显示，虚拟地址则默认显示。通常虚拟地址以“[区域名称]:[虚拟地址]”格式显示，如“.text:0000000000401530”。

显示窗口的左边部分称为“箭头窗口”，用于描述函数中的非线性流程。实线箭头表示非条件跳转，虚线箭头则表示条件跳转。如果一个跳转将控制权交给程序中的某个地址，这时会使用粗线，出现这类逆向流程，通常表示程序中存在循环。

在反汇编窗口区按空格键，可以切换为流程图视图模式，如图6.18所示。

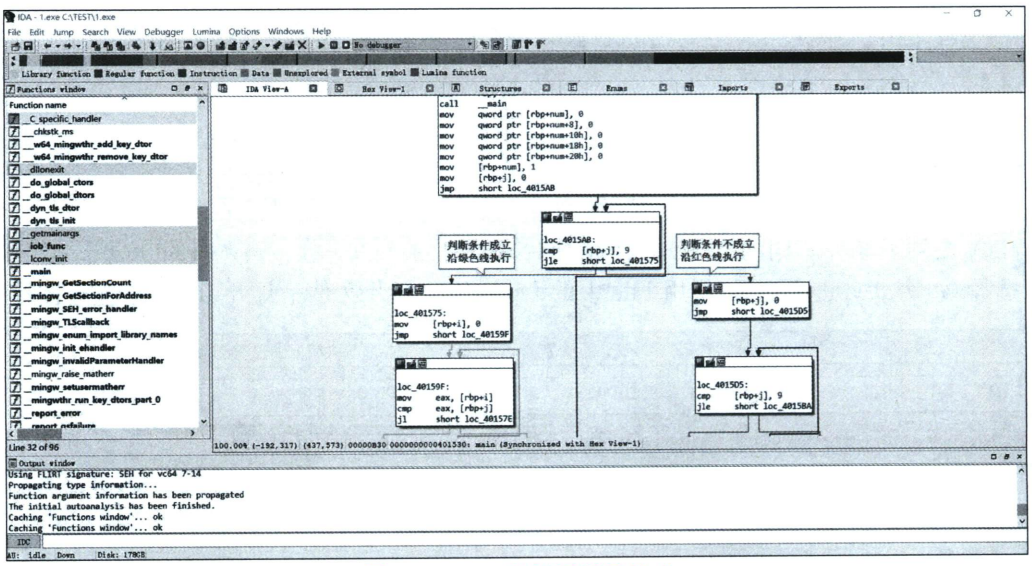


图6.18 IDA流程图视图模式

流程图视图将一个函数分解为许多基本块，与程序流程图类似，生动地显示该函数由一个块到另一个块的控制流程。在屏幕上可以发现，IDA使用不同的彩色箭头线区分函数之间各种类型的流：绿色箭头线代表判定条件成立，红色箭头线代表判定条件不成立，蓝色箭头线表示指向下一个即将执行的块。

通过选择菜单“View”→“Open subviews”选项可以打开更多窗口，这里介绍一些经常用到的窗口和功能。

1. Names窗口

Names窗口简要列举了一个二进制文件的所有全局名称，名称是对一个程序虚拟地址的符号描述。在最初加载文件的过程中，IDA会根据符号表和签名分析派生出名称列表。用户可以通过Names窗口迅速导航到程序列表中的已知位置。双击Names窗口中的名称，会立即跳转到显示该名称的反汇编视图。Names窗口如图6.19所示。

Names窗口显示的名称采用了颜色和字母编码，其编码方案总结如下。

(1) F：常规函数。

(2) L：库函数。

(3) I：导入的名称，通常为共享库导入的函数名称。

(4) D：数据。已命名数据的位置通常表示全局变量。

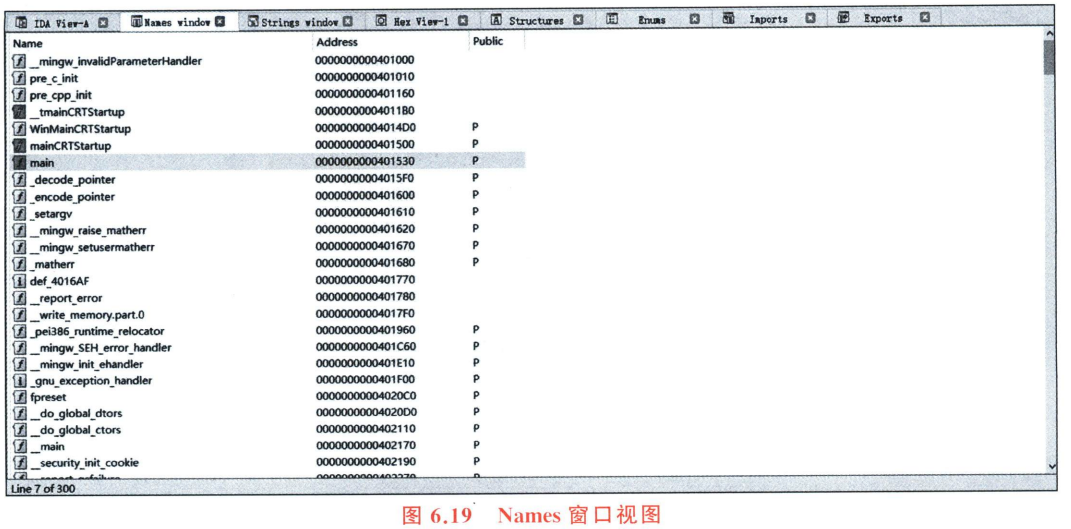


图 6.19 Names 窗口视图

(5) A：字符串数据。

2. Strings 窗口

Strings 窗口功能在 IDA 5 及以前的版本是默认打开的窗口，新版本已经不再默认打开，但是可以通过选择“View”→“Open subviews”→“Strings”选项来打开。打开后窗口如图 6.20 所示。

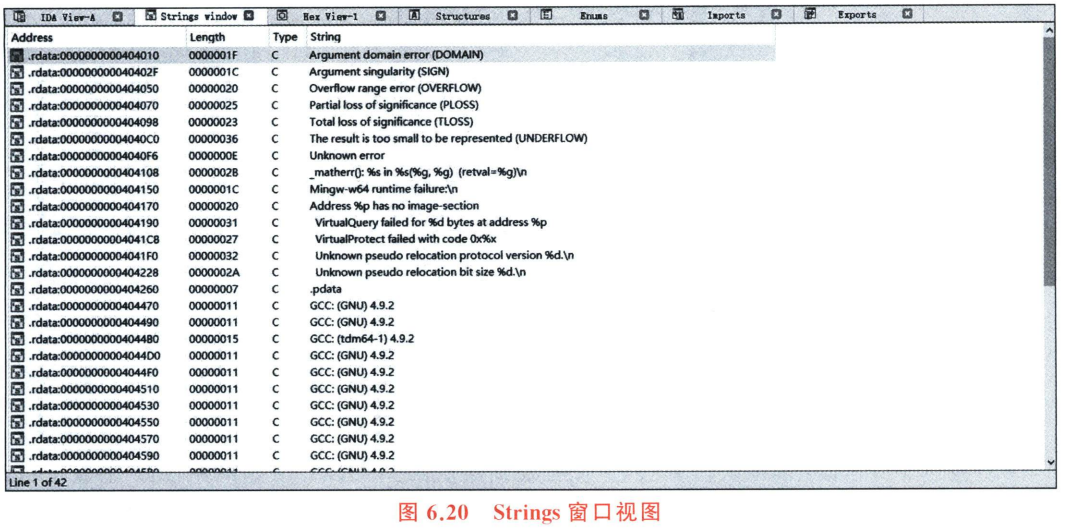


图 6.20 Strings 窗口视图

Strings 窗口中显示的是从二进制文件中提取出的一组字符串，以及每个字符串所在的地址。双击 Strings 窗口中任何字符串，汇编代码窗口将跳转到该字符串所在的地址。将 Strings 窗口与交叉引用结合，可以迅速定义感兴趣的字符串，并追踪到程序中的任何引用该字符串的位置。

通过此窗口可以看到程序中所有的常量字符串列表，逆向分析一个程序从字符串入手是一个方向。也可以通过 Alt+T 快捷键打开，可以通过此窗口查找某个指定的字符串，如图 6.21 所示。

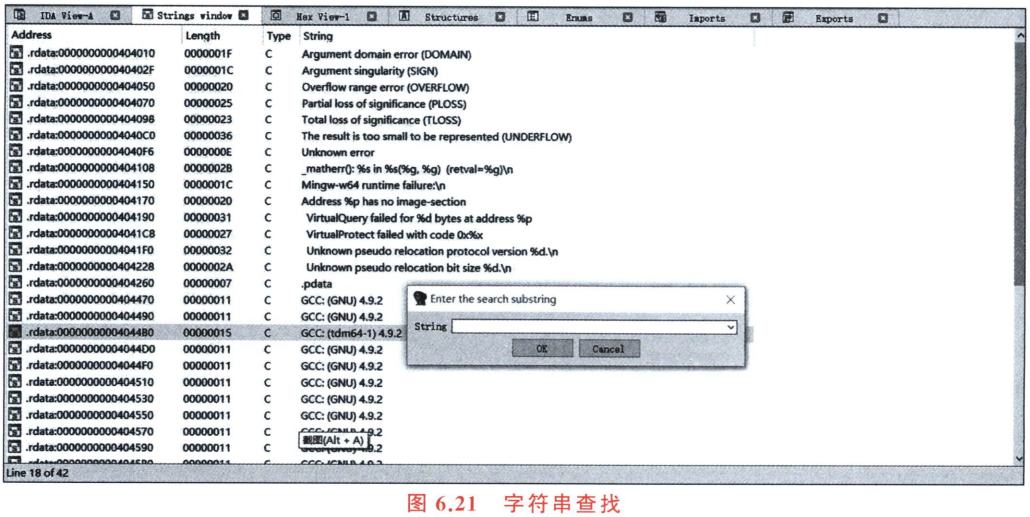


图 6.21 字符串查找

3. Function call 窗口

Function call 窗口显示所有函数的调用关系，如图 6.22 所示。光标放在某一主调或被调函数条目上，即可显示相关反汇编代码。

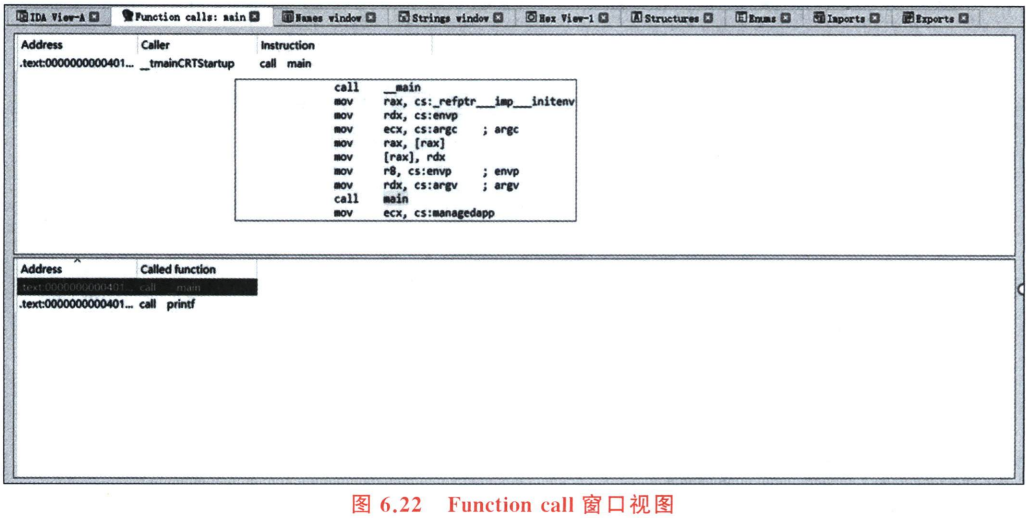


图 6.22 Function call 窗口视图

除了通过 Function call 窗口列表显示函数调用关系外，IDA 还提供了图形化显示方法。在 IDA View 窗口的反汇编代码中找到函数名，右击函数名选择 Xref graph to 选项，可以看到该函数被其他函数调用的信息，右击函数名选择 Xref graph from 选项，可以看到该函数调用的其他函数，分别如图 6.23 和 6.24 所示。

4. 地址跳转

通过菜单选择 Jump→Jump to address 选项或直接输入 G 进行地址跳转，使用此项功能可以跳转到指定地址的汇编代码段，如图 6.25 所示。

此外，IDA 还支持执行自动化的脚本进行静态分析，提供动态调试功能等，已经成为大多数人公认的分析恶意代码的标准并迅速成为攻击研究领域的重要工具。IDA 可供反病毒公司、漏洞研究公司、大型软件开发公司等用于恶意代码分析、漏洞研究、隐私保护及其他学术研究，是安全分析人士不可缺少的利器。

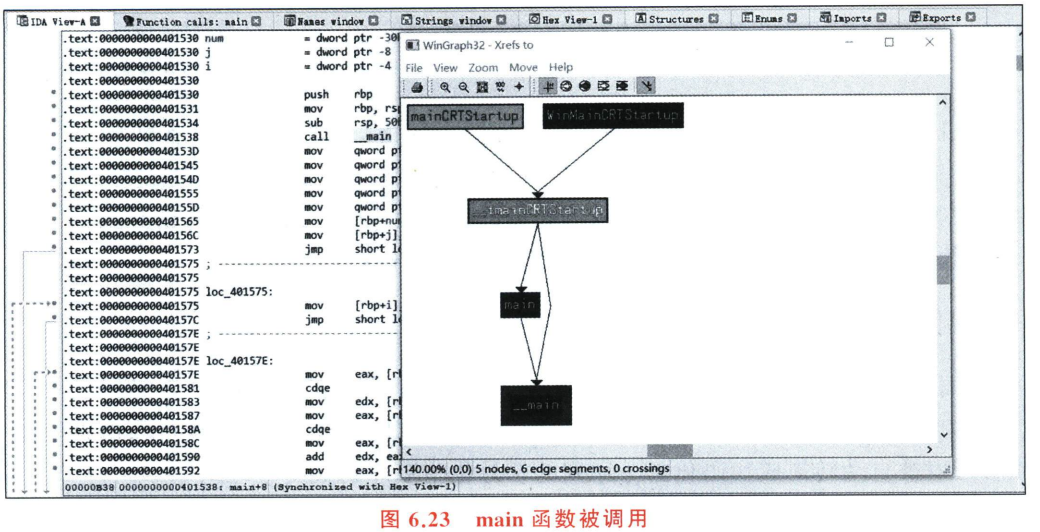


图 6.23　main 函数被调用

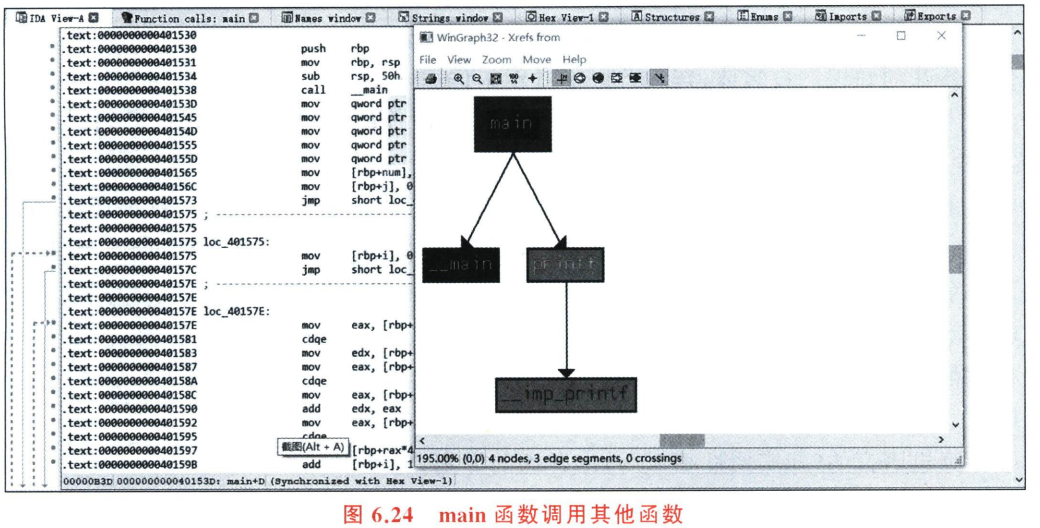


图 6.24　main 函数调用其他函数

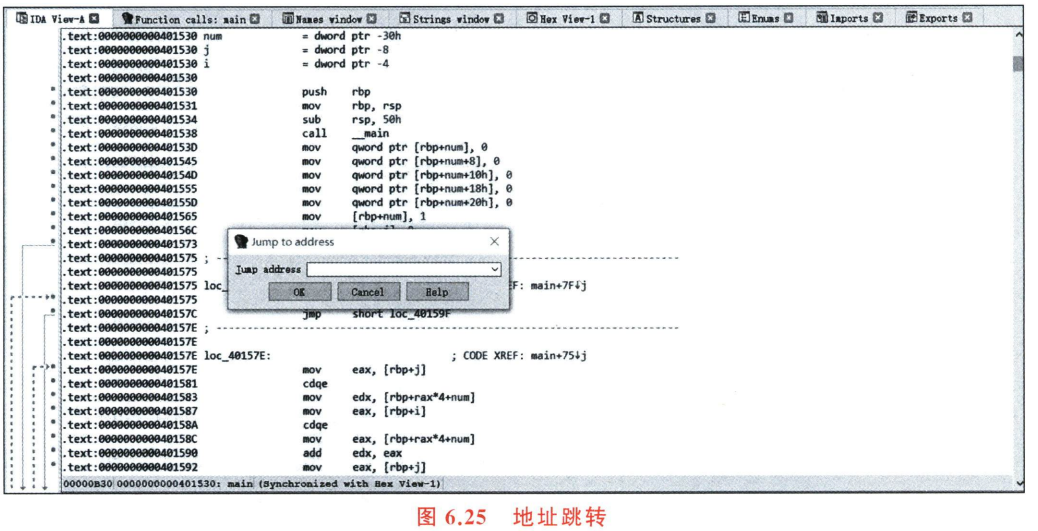


图 6.25　地址跳转

6.5 软件知识产权保护

计算机软件是人类知识、经验、智慧和创造性劳动的成果，具有知识密集和智力密集的特点，是一种非常典型的知识产权。然而计算机软件相较于其他商用产品有开发周期长、成本高、易复制等缺点，在我国，相关软件产品的知识产权保护也相对薄弱。在这种形势下，加强针对计算机软件类相关产品的知识产权保护显得十分重要和紧迫。

6.5.1 软件知识产权

在我国计算机软件包括计算机程序及其文档。我国《计算机软件保护条例》中规定，计算机程序是为了得到某种结果而可以由计算机等具有信息处理能力的装置执行的代码化指令序列，或者可以被自动转换成代码化指令序列的符号化指令序列或符号化语句序列，包括了源程序和目标程序。文档是用来描述程序的内容、组成、设计、功能规格、开发情况、测试结果及使用方法的文字资料和图表等，如程序设计说明书、流程图、用户手册等。

计算机软件作为人类的智力劳动成果，毫无疑问应该成为知识产权的保护对象。计算机软件作为特殊的知识产权客体，存在着以下有别于传统知识产权客体的特征。

(1) 计算机软件具有作品属性、技术性特征和商业价值，所以计算机软件可以兼有著作权、专利权和商业秘密三种属性。

(2) 计算机软件著作权中的财产权具有不完整性。传统的文字作品具有翻译权，如果将一种文字的作品翻译为另外一种文字，需要征得作者的同意并支付相应的费用。但是计算机软件是数字作品，没有这项权利，不同计算机语言之间的“抄袭”不应认定侵犯著作权。

(3) 计算机软件的专利权带有较强模糊性。与传统的专利权客体不同，计算机软件的专利侵权大都体现在无形的计算机语言上。对于实现特定功能而言，相同的计算机语言可以有不同的表达，不同的计算机语言也可以实现完全相同的功能。虽然学界提出了“等同原则”，以期对于计算机语言的“非实质性相似”做出合理的界定，但是因该原则在理论上存在不同的适用原则，因而并没有消除计算机软件专利权的这种模糊性特征。

(4) 计算机软件商业秘密保护难度较大，反向工程和开源运动都会对计算机软件的商业秘密形成冲击。另外，软件的形成过程都不同程度地带有一定的信息，究竟所带有的信息量达到何种程度才算商业秘密，目前尚有争议。因此计算机软件的商业秘密的保护难度比传统的商业秘密大。

总之，计算机软件符合知识产权的基本特征，具备知识产权的基本属性，但是鉴于计算机软件本身特殊的技术性，又有着与传统知识产权客体不同的内在属性与外部特征。计算机软件作为高科技产业，其技术更新之快可谓日新月异，计算机软件的法律保护对于传统的知识产权理论提出了挑战。

6.5.2 软件知识产权法律保护

软件的开发研制是一种高强度的脑力劳动，是人类智慧的结晶，是一种无形的知识产品。软件开发具有开发成本高昂，人力物力投入巨大的特点，但其复制极易掌握，且费用低廉，因此越来越多的不法分子通过销售盗版软件来牟取暴利。这严重挫伤了软件开发者开发软件产品的积极性，损害了其正当利益，阻碍了整个软件产业的发展。因此，如何在客观上以法律手段

保护计算机软件不被侵权，成为各国政府及其产业界共同谋求解决的难题。

版权法和专利法是计算机软件保护的最重要的两种方式，也是计算机软件知识产权保护的国际发展趋势。除此以外，计算机软件还可以商业秘密法、反不正当竞争法、合同法、与贸易有关的知识产权协定，以及其他非法律的保护方式予以保护。下面分别介绍我国几种软件知识产权保护模式。

1. 软件版权法律保护

我国对于计算机软件的保护主要依照著作权的保护方式进行，目前我国在软件相关产品的著作权保护方面，逐步形成了以《中华人民共和国著作权法》(下面简称《著作权法》)为主体，《计算机软件保护条例》(下面简称《条例》)与我国加入的《伯尔尼公约》《世界知识产权组织版权公约》及《与贸易有关的知识产权协定》相配套的著作权法律保护体系。

1990年通过的《著作权法》列举了受著作权保护的作品范围，软件是其中一项。只要是具有独创性的软件，不论是否具有新颖性都受到著作权法的保护。为了对计算机软件进行更加专门细致的保护，1991年，国务院以《著作权法》为基础，制定了《条例》，之后又颁布了《计算机软件登记办法》，针对软件保护进行了专门规定。1995年国家著作权局制定的《关于不得使用非法复制的计算机软件的通知》明确规定："禁止任何机构和个人在其计算机系统中使用未授权的计算机软件"。

我国在2011年加入世界贸易组织后，先后多次修订《条例》和《著作权法》，满足国内外软件法律保护与国际接轨的需求的同时，也建立起体系化的软件知识产权保护制度。根据《条例》，未经著作权人的许可，其他人不得对软件进行复制、抄袭等侵权行为，一定程度上保护了著作权人的合法权益，调整计算机软件在开发、传播和使用过程中发生的利益关系，鼓励计算机软件的开发与应用，促进软件产业和国民经济信息化的发展。

2. 软件专利法律保护

《著作权法》保护的主要是产品的外在表达内容，而对于计算机软件产品的内在想法，即软件的核心算法思想等无法提供有效的保护，因此国内外均采用了申请软件专利等来保障软件开发者的权益。

2021年6月1日开始实施的《中华人民共和国专利法》(下面简称《专利法》)，在第二十五条第(二)款规定对于智力活动的规则和方法不授予专利权。如果发明专利申请仅涉及纯软件，或是记录在机器可读介质载体上的程序，则不论它以何种形式出现，都属于智力活动的规则和方法，因而不能申请专利。但是，如果一件含有计算机程序的发明专利申请能完成发明目的，并产生积极效果，构成一个完整的技术方案，也不应仅因为该发明专利申请含有计算机程序，而判定为不可以申请专利。

由此，国家知识产权局专利局在《专利法》和《专利法实施细则》基础上，制定并发布了《专利审查指南》。该指南对于计算机软件发明和创新方面的专利申请，提出了需要满足的四个条件：一是针对涉及计算机内部程序运行进行性能和效率改善的计算机程序；二是针对计算机软件外第三方数据处理的计算机程序；三是涉及工业过程中使用的计算机程序；四是用于测试和测量过程控制的计算机程序。该指南通过规定专利计算机软件产品需要公开全部技术方案，其中包括逻辑框图、设计流程等核心部分，从而避免了在实际软件产品生产过程中，对他人著作的软件进行反向操作等行为。

3. 软件商业秘密法律保护

商业秘密作为一种新型知识产权，在当前实践中由专门的知识产权法院审理侵犯商业秘

密的案件。与其他知识产权相仿，商业秘密也具有可复制性、可转让性和一定程度的专有性等。但相较于一般的知识产权，商业秘密也有其特殊之处，在不公开的状态下，不需要向相关部门申请，也可以受到法律的保护，并且在这种情况下，更能显示知识产权巨大的商业利益、社会效益，使得权利人在竞争中处于强势地位。

商业秘密包括技术秘密、经营管理经验和其他关键性信息。计算机软件产品必须符合一定的构成条件，才能够被视为商业秘密，其知识产权才能受到法律保护。就计算机软件行业来说，商业秘密是关于当前和设想中的产品开发计划、功能和性能规格、算法模型、设计说明、流程图、源程序清单、测试计划、测试结果等资料；也可以包括业务经营计划、销售情况、市场开发计划、财务情况、顾客名单及其分布、顾客的要求及心理、同行业产品的供销情况等。

我国2019年修订的《反不正当竞争法》第9条规定了侵犯商业秘密的行为包括：盗窃、贿赂、欺诈、胁迫；披露、使用或允许他人使用；违约或违反保密的要求，披露、使用或者允许他人使用其所掌握的商业秘密。对于那些达不到申请专利条件又无法对其核心设计进行有效的著作权保护的计算机软件等相关产品，商业秘密便成为一个重要补充手段。

6.5.3 软件版权技术保护

在软件侵权行为中，对于一些侵权主体比较明确的，可以通过法律手段解决。但是对于一些侵权主体比较隐蔽或分散的，受到时间、人力和财力诸多因素制约，还不能进行全面管制。为保护软件所有人的权利，必须从技术、法律等各方面考虑，进行全面保护。

软件版权保护技术通过技术手段进行软件保护，防止对软件产品的非法复制和使用，以及对软件产品进行的非法修改。软件版权保护技术可以分为基于硬件的和基于软件的两大类。

6.5.3.1 基于硬件的版权保护技术

基于硬件的版权保护技术的核心是在软件授权加密的过程中引入硬件，利用硬件技术的安全性为软件产品的安全性提供保障。其技术原理是为软件的运行或使用关联一个物理介质或物理模块，其中包含一个秘密信息（如序列号、一段代码或密钥），并使得这个秘密信息不易被复制、篡改和观察分析。

基于硬件的保护方法主要有光盘保护、加密狗保护、可信计算芯片保护等。通常硬件加密成本较高，主要用于行业用户中，不适用于普通用户，也不利于网上发布软件。

6.5.3.2 基于软件的版权保护技术

基于软件的保护方式因为其丰富的技术手段和优良的性价比，目前仍然是市场主流的软件版权保护技术。目前常用的技术有注册验证、软件水印、代码混淆、软件加壳等。

1. 注册验证

基于注册验证的版权控制是使用比较广泛的软件版权保护方式。软件供应商对用户信息进行加密，密文作为注册信息返回给用户。软件运行时，通过验证注册信息确认软件是否授权，实现保护。

注册信息一般基于软件使用者的用户信息生成，可以是一串序列号或注册码，也可以是一个授权文件或其他存在形式。在验证比较严格的情况下，用户信息应该包含标志用户唯一性且不可复制的信息，如计算机的CPU、硬盘等关键部件的硬件序列号等。

对合法授权软件，软件供应商会提供一个注册文件，该文件包含加密过的用户信息、注册码等注册信息。用户只要将该文件放入指定的目录下，软件每次启动时，从该文件中读取注册

信息并按设计好的算法进行处理，然后与用户输入的信息或注册码进行比较，根据比较结果判断是否为授权软件，如果是则以正版软件模式来运行。

2. 软件水印

软件水印是在软件产品中嵌入版权保护信息和身份认证信息，并通过检测出的版权保护信息和身份认证信息来确认是否拥有产品的版权，防止软件盗版和非法复制。软件水印具有信息隐藏的功能，水印信息可以包含作者、发行商、所有者、使用者等，并携带有版权保护信息和身份认证信息，在设计水印时对这些信息加密，可以防止水印信息遭到破坏或修改。

软件水印技术包括水印信息的生成、水印嵌入和水印提取三个过程。根据软件水印的嵌入和提取技术来分，可分为静态水印和动态水印。静态水印把水印信息存储在可执行程序代码中。比如，放在安装模块部分、指令代码部分，或者是调试信息的符号部分。软件运行时准确提取软件中的加密水印信息，解密后与原始信息对比，验证软件的版权信息。动态水印则是把水印信息保存在程序的执行状态中，而不是程序源代码本身。软件开发过程中，选择一种动态水印算法，和正常的软件模块一起编写、编译。软件执行过程中要求输入特定的序列，根据输入，程序会运行到某种状态，这些状态就代表水印，通过这些水印验证软件的版权。

3. 代码混淆

代码混淆技术是一种重要的软件版权保护方法。代码混淆指对拟发布的应用程序进行保持语义的变换，使得变换后的程序和原来的程序在功能上相同或相近，防止攻击者通过静态分析和逆向工程来分析编程者的思想，获取机密数据和核心算法。根据混淆原理和对象不同，代码混淆技术又可以分为布局混淆、控制结构混淆、数据混淆、预防混淆等。

代码混淆的实质是在原始程序代码中嵌入一段伪代码，伪代码与原有代码组合在一起就能增加攻击的难度。从混淆后的程序中完全恢复出混淆前的原始程序通常相当困难，甚至是不可能的。当然，攻击者也可以利用某些攻击方法取得程序部分甚至全部有用信息。

尽管已有理论证明了在终端运行且没有硬件辅助保护机制的代码混淆技术不可能提供彻底的保护，但是在实际应用中，代码混淆并不需要对软件提供绝对的保护，只要能使攻击者付出极大代价，就认为达到目的了。

4. 软件加壳

“壳”是计算机软件中一段专门负责保护软件不被非法修改或反编译的程序，它先于软件原程序运行并拿到控制权，进行一定处理后再将控制权转交给原程序，实现保护软件的任务。加壳后的程序能够有效防范静态分析和增加动态分析的难度。

根据对原程序实施保护方式的不同，壳大致可以分为以下两类。第一类是压缩保护型壳，即对原程序进行压缩存储的壳。这种壳以减小原程序的体积为目的，在对原程序的加密保护上并没有做过多的处理，所以安全性不高，很容易脱壳。第二类是加密保护型壳，即根据用户输入的密码以相应的算法对原程序进行加密。当程序执行的时候会提示用户输入口令或注册码。如果破解者强行更改密码检测指令，因为被加密的代码并没有用相应的口令进行解密，程序还没有被还原，会导致程序不正确地执行。

在基于注册验证的版权控制中结合采用加密保护型壳，将注册信息作为原程序解密的密码，对注册验证逻辑的修改将导致原程序解密过程的失败，这样可以大大提高基于软件注册的版权控制策略的入口安全性。

6.6 本章小结

软件是信息系统的核心基础，软件安全无疑是信息安全保障的关键。软件安全涉及软件开发生命周期多个阶段，应当采取安全开发和测试措施，确保软件在恶意攻击下仍然正常运行。本章首先对软件安全现状、软件缺陷和漏洞及恶意代码威胁进行了概述；其次基于对传统软件开发中安全局限性的阐述，对安全软件开发生命周期和模型进行了介绍；然后对恶意代码分类、原理、分析技术分别进行阐述；接下来对黑盒测试、白盒测试等传统软件测试方法和模糊测试、渗透测试等软件安全测试方法，以及静态分析工具 IDA PRO 进行了详细介绍；最后针对软件侵权问题，分别从法律和技术两个层面讨论软件知识产权保护方法。

思 考 题

1. 什么是软件安全？
2. 简要介绍几个主要的传统软件开发生命周期模型。
3. 什么是安全软件开发生命周期？其主要组成部分是什么？
4. 简述微软可信计算安全开发生命周期。
5. 计算机病毒、蠕虫、木马这三种恶意软件的区别是什么？
6. 简述恶意代码的基本原理。
7. 恶意代码的生存技术都有哪些？
8. 试比较针对恶意代码的静态分析技术和动态分析技术。
9. 恶意代码分析工具都有哪些？如何分类？各类的作用如何？
10. 传统软件测试方法都有哪些？各有什么特点？
11. 试比较白盒测试、黑盒测试和灰盒测试方法。
12. 模糊测试和渗透测试分别有什么作用？
13. 我国对于软件知识产权保护有哪些法律保护途径？
14. 试比较软件版权保护技术。